

华中科技大学

课程实验报告

课程名称： 数据结构实验

专业班级 计算机启明 2401

学 号 U202414877

姓 名 刘羽桐

指导教师

报告日期 2025 年 5 月 27 日

计算机科学与技术学院

目 录

1 基于链式存储结构的线性表实现.....	1
1.1 问题描述	1
1.2 系统设计	1
1.3 系统实现	5
1.4 系统测试	14
1.5 实验小结	19
2 基于邻接表的图实现	20
2.1 问题描述	20
2.2 系统设计	20
2.3 系统实现	25
2.4 系统测试	42
2.5 实验小结	46
3 课程的收获和建议	47
3.1 基于链式存储结构的线性表实现	47
3.2 基于邻接表的图实现.....	47
参考文献	48
附录 A 基于顺序存储结构线性表实现的源程序	49
附录 B 基于链式存储结构线性表实现的源程序	68
附录 C 基于二叉链表二叉树实现的源程序.....	89
附录 D 基于邻接表图实现的源程序.....	114

1 基于链式存储结构的线性表实现

链式存储结构是实现线性表的非顺序存储结构。本次实验主要实现了链式线性表的基本构筑与操作，通过一个集成的系统封装了相关操作，并提供了一个简单的用户界面。

1.1 问题描述

本实验主要解决以下几个问题：

- 链式线性表的基本和数据结构设计
- 链式线性表的创建、销毁、查找、删除、插入等基本操作
- 链式线性表的进阶操作功能以及线性表的文件输入输出
- 链式线性表操作系统的构筑以及多链表显示管理

下面从系统总体设计构筑以及系统具体功能实现两个方面来解决上述问题

1.2 系统设计

1.2.1 数据结构设计

本实验涉及的数据结构主要有两种：线性表节点以及多链表结构。其中线性表节点用于基本的线性表构建及操作；多链表结构用于实现多链表管理。

1) 节点

单个链式线性表的定义相较于顺序表更为简单，只需要定义一个节点结构体即可。每个节点包含数据域和指向下一个节点的指针域。具体定义如下：

```
1 typedef struct LNode {  
2     ElemType data;  
3     struct LNode *next;  
4 } LNode, *LinkList;  
5
```

Listing 1 链表节点定义

其中，*ElemType* 是数据域的类型，可以根据需要定义为任意类型，*LNode* 是链表节点的结构体，*LinkList*，用于指代整个链表。

2) 多链表结构

多链表结构用于管理多个链式线性表。其在本质上是一个链表头节点数组，每个头节点指向一个链式线性表的头节点。此外，该结构存储了对应链表的名称，利用索引一一对应。具体定义如下：

```
1 typedef struct LISTS {
2     LinkList *elem = NULL;
3     char **name;
4     int length;
5     int listsize;
6 } LISTS;
7
```

Listing 2 多链表结构定义

其中，‘elem’是指向链表头节点的指针数组，‘name’是存储链表名称的字符串数组，‘length’是当前链表数量，‘listsize’是当前多链表的最大容量。

系统功能设计 用户可以通过系统进行的操作可以分为两个方面：链式线性表的基本操作以及多链表的管理。具体设计架构如下：

3) 单链表操作状态机

单链表操作状态机主要包括以下几个状态：创建、销毁、插入、删除、查找、显示等。每个状态对应一个函数，用户可以通过输入指令来切换状态并执行相应的操作。具体来说，程序进入状态机后，将进入一个循环，等待用户输入命令，根据输入的命令执行相应的操作。每个操作完成后，系统会返回到主菜单状态，等待下一次用户输入，直到用户输入退出指令。具体结构如下：

```
1 while(op)
2 {
3     /*
4         Operating Menu
5     */
6     scanf("%d", &op);           // 获取用户输入的操作码
7     if(op == 0)
8     {
9         printf("Returning to Main Menu...\n");
10        op = 1;
11        break;                 // 如果用户输入0，则返回
12    }
13    switch(op)
14    {
15        case 1:
16        case 2:
```

```
17     ...
18 }
19
20 printf("Press any key to continue...");
21 getchar(); // 清除缓冲区中的换行符
22 getchar(); // 等待用户输入
23 }
24
25 op = 1; // 重置操作码，返回主菜单
26
```

Listing 3 单链表操作状态机

4) 多链表管理状态机

多链表管理状态机的核心在于如何实现对任意一个链表的创建，销毁以及实现所有单链表的具体操作，同时要在链表切换之后能够保留之前的操作状态。此外，为了减少代码的复杂度，实现单链表操作和多链表操作的代码复用，系统采用了一个指针管理的模式，每次进入单链表管理部分之前会读取当前待操作链表 `L_operated` 对应的实际链表指针，实际操作过程中只使用 `L_operated` 作为函数入口的操作对象。具体结构如下：

```
1  if(mode == 2)
2  {
3      system("cls");
4      printf("Multi-Lists Operation Mode:\n");
5      if(Lists.elem[list_idx] == NULL )
6      {
7          L_operated = NULL;
8          printf("No lists exist!\n");
9          // 如果当前链表为空，则给予提示
10         }
11         else
12         {
13             L_operated = &Lists.elem[list_idx];
14             printf("List id: %d\n",list_idx + 1);
15         }
16     }
```

Listing 4 多链表管理状态机

如上述代码所示，系统首先判断当前是否处于多链表管理模式，如果是，则获取当前待操作链表的指针，具体来说是通过获取 `main` 函数中的全局变量 `list_idx` 来获取当前待操作链表的索引，如果要切换链表，只需修改 `list_idx` 即可。

5) 最终效果

最终效果如图：1-2所示，用户可以通过输入指令来进行链表的创建、销毁、插入、删除、查找等操作，同时可以通过多链表管理状态机来管理多个链式线性表。

```
Multi-Lists Operation Mode:
-----
Current list: list4    List id: 3
-----
Menu for Linear Table On Link Structure
-----
1. InitList      7. LocateElem
2. DestroyList   8. PriorElem
3. ClearList     9. NextElem
4. ListEmpty     10. ListInsert
5. ListLength    11. ListDelete
6. GetElem       12. ListTraverse
-----
File Operation Functions
-----
13. SaveList     14. LoadList
-----
Additional Functions
-----
15. ReverseList  16. SortList
17. InitSequenceList
-----
Multi-Lists Functions
-----
18. AddList      21. LocateList
19. RemoveList   22. PrintLists
20. SwitchList   23. InitMultilists
-----
0. Return to Main Menu
-----
Choose your operations [0~21]:
```

图 1-1 链式线性表操作系统界面

1.3 系统实现

完成系统架构介绍后，下文将介绍系统的具体实现，包括各个主要函数的实现思想，复杂函数可辅助流程图进行说明，函数和系统实现的源代码放在附录中。

本系统实现了以下功能：

- 链表初始化
- 链表销毁
- 链表清空
- 链表判空
- 链表长度
- 元素定位
- 元素获取
- 获取前驱，后继元素
- 插入元素
- 删除元素
- 链表遍历
- 链表读写
- 链表反转
- 逆序获取元素
- 链表排序
- 添加链表
- 移除链表
- 切换链表
- 定位链表
- 输出链表清单

下面具体介绍各个功能的实现方式

1) 链表初始化

链表初始化函数主要用于创建一个新的链表，并返回创建状态。具体实现如下：

```
1  status InitList(LinkList *L)
2  // 线性表L不存在，构造一个空的线性表，返回OK，否则返回
   INFEASIBLE。
3  {
4      if(*L == NULL)
5      {
6          *L = (LNode *)malloc(sizeof(LNode));
7          (*L) -> next = NULL;
8          return OK;
9      }
10
11     return INFEASIBLE;
12 }
13
```

Listing 5 链表初始化

2) 链表销毁

链表销毁可以视为链表创建的逆过程，遍历链表的每一个节点并释放内存即可，注意要先访问下一个节点再释放当前节点空间，代码见附录

3) 链表清空

链表清空函数主要用于将链表中的所有元素数据清空，类似链表销毁函数，但不释放内存空间，代码见附录

4) 链表判空

链表判空函数主要用于判断链表是否为空，只需判断链表头节点的指针是否为 NULL 即可，代码见附录

5) 链表长度

链表长度函数主要用于计算链表中元素的个数，遍历链表并计数即可，代码见附录

6) 元素获取

本质是进行线性表的遍历，读取第 i 个元素的值，不过要注意的是 i 越界的情况，可以直接在函数体内做一次遍历的判别即可，也可以先调用链表长度函数获取链表长度再进行判断，代码如下：

```
1  status GetElem(LinkList *L, int i, ElemType *e)
2  {
3      if(*L == NULL)
4          return INFEASIBLE;
5
6      LNode * temp = *L;
7      int count = 0;
8      while(temp -> next != NULL && count < i)
9      {
10         temp = temp -> next;
11         count++;
12     }
13
14     if(count < i - 1 || count == 0) return ERROR;    // i越界
15
16     *e = temp -> data;
17
18     return OK;
19 }
```

Listing 6 元素定位

7) 元素定位

元素定位函数主要用于查找链表中是否存在某个元素，遍历链表并比较元素值即可，代码见附录

8) 前驱, 后继元素获取

前驱, 后继元素获取函数主要用于获取某个元素的前驱和后继元素, 遍历链表并记录当前元素的前一个和后一个元素即可, 采用双指针法, 代码如下:

```
1  status PriorElem(LinkList *L, ElemType e, ElemType *pre)
2  {
3      if(*L == NULL)
4          return INFEASIBLE;
5
6      // 如果链表为空(仅有头节点)
7      if((*L)->next == NULL)
8          return ERROR;
9
10     // 双指针法查找前驱元素
11     LNode *current = (*L)->next;
12     LNode *prior = *L;
13
14     while(current != NULL)
15     {
16         if(current->data == e && prior != *L)
17         {
18             *pre = prior->data;
19             return OK;
20         }
21         prior = current;
22         current = current->next;
23     }
24
25     // 如果没找到元素e或元素e是第一个元素(无前驱)
26     return ERROR;
27     /***** End *****/
28 }
29
```

Listing 7 前驱, 后继元素获取

后继元素的获取类似, 只需将指针顺序调整即可, 代码见附录

9) 插入元素

插入元素的实现主要在于定位前驱节点和后继节点, 然后将新节点插入到前驱和后继之间, 注意越界的情况, 代码实现如下:

```
1  status ListInsert(LinkList *L, int i, ElemType e)
2  {
3      if(*L == NULL)
4          return INFEASIBLE;
5
6      int count = 0;
7      LNode *prior = *L;
```

```
8      LNode *current = (*L) -> next;
9      while(current != NULL && count < i - 1)
10     {
11         prior = current;
12         current = current -> next;
13         count++;
14     } // 遍历到第i-1个节点，获
    取前驱节点prior和后继节点current
15
16     if(count != i - 1) return ERROR; // i越界
17
18     //插入元素
19     LNode *insert = (LNode *)malloc(sizeof(LNode));
20     insert -> data = e;
21     prior -> next = insert;
22     insert -> next = current;
23     return OK;
24     /***** End *****/
25 }
26
```

Listing 8 插入元素

10) 删除元素

类似于插入元素的实现，删除元素的实现主要在于定位前驱节点和后继节点，然后将前驱节点指向后继节点，注意越界的情况，代码见附录。

11) 链表读写

链表的文件读写本质上只是链表的遍历，注意文件读和写的格式一致性即可。在格式上，由于链表不需要预分配空间，因此也不需要记录链表的长度，只要记录各个节点的值即可，下面展示链表读取的代码

```
1      status LoadList(LinkList *L, char FileName[])
2      {
3          if(*L != NULL) // 如果链表已存在，返回INFEASIBLE
4          {
5              printf("Lists already exist, rewrite it? (y/n): ");
6              char choice;
7              scanf(" %c", &choice);
8              if(choice != 'y' && choice != 'Y')
9                  return INFEASIBLE; // 用户选择不重写
10         }
11
12         FILE *fin = fopen(FileName, "r");
13         if(fin == NULL)
14             return ERROR;
15
16         DestroyList(L); // 销毁原有链表
17     }
```

```
18 // 创建头节点
19 *L = (LinkedList)malloc(sizeof(LNode));
20 if(*L == NULL) {
21     fclose(fin);
22     return ERROR;
23 }
24 (*L)->next = NULL;
25
26 LNode *tail = *L; // 尾指针, 用于尾插法
27 int value;
28
29 // 读取文件中的数据并创建链表节点
30 while(fscanf(fin, "%d", &value) == 1)
31 {
32     LNode *newNode = (LNode*)malloc(sizeof(LNode));
33     if(newNode == NULL) {
34         fclose(fin);
35         // 释放已分配的链表
36         LNode *current = *L;
37         while(current != NULL) {
38             LNode *temp = current;
39             current = current->next;
40             free(temp);
41         }
42         *L = NULL;
43         return ERROR;
44     }
45
46     newNode->data = value;
47     newNode->next = NULL;
48     tail->next = newNode;
49     tail = newNode;
50 }
51
52 fclose(fin);
53 return OK;
54 /***** End 2 *****/
55 }
56
```

Listing 9 链表读取

注意链表读取的过程相当于重复了一边链表创建的过程

12) 链表反转

链表反转的实现主要是将每个节点的指针域指向前一个节点。具体的算法是在遍历链表的过程中，使用三个指针分别指向当前节点、前一个节点和后一个节点，然后逐个修改指针的指向，直到遍历完整个链表。修改方式如图1-2所示。

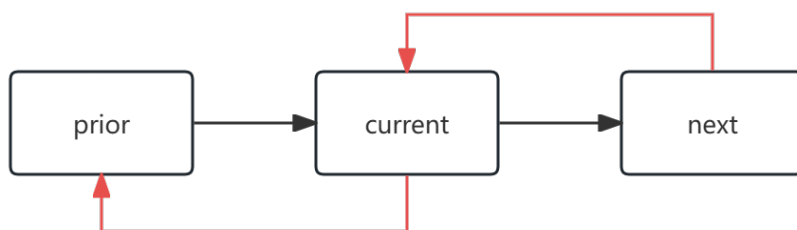


图 1-2 链表反转示意图

代码实现如下：

```
1  status reverseList(LinkList *L)
2  {
3      if(*L == NULL)
4          return INFEASIBLE;
5
6      LNode *current = (*L)->next->next; // 从第一个数据节点开
   始
7      LNode *prev = (*L)->next;
8      LNode *next = NULL;
9
10     while(current != NULL)
11     {
12         next = current->next;           // 图1-3-1操作对应代
   码
13         current->next = prev;
14         prev = current;
15         current = next;
16     }
17
18     (*L)->next->next = NULL;           // 尾节点处理
19     (*L)->next = prev;
20
21     return OK;
22 }
23
```

Listing 10 链表反转

13) 逆序删除元素

逆序获取元素的实现主要是通过链表反转的方式实现，先将链表反转，然后再按照顺序获取元素即可，但相对而言有些冗余。从另一个角度看，获取倒数第 n 个元素可以认为是获取最后一个元素的 n -前驱节点，因此只要设置步长差距为 $n-1$ 的快慢指针即可，代码实现如下：

```
1  int DeleteNodefromEnd(LinkList *L, int n)
2  {
3      if(*L == NULL || (*L)->next == NULL)
4          return INFEASIBLE;
5
6      if(n <= 0)
7          return ERROR; // 添加参数检查
8
9      LNode *dummy = *L; // 使用头节点作为虚拟节点
10     LNode *fast = dummy;
11     LNode *slow = dummy;
12
13     // fast先走n步（注意是n而非n-1）
14     for(int i = 0; i < n; i++)
15     {
16         if(fast->next == NULL)
17             return ERROR; // n大于链表长度
18         fast = fast->next;
19     }
20
21     // fast和slow一起走，直到fast到达链表末尾
22     while(fast->next != NULL)
23     {
24         slow = slow->next;
25         fast = fast->next;
26     }
27
28     // 此时slow指向要删除节点的前一个节点
29     ElemType e = slow->next->data; // 要删除节点的值
30     LNode *temp = slow->next;
31     slow->next = slow->next->next;
32     free(temp);
33
34     return e;
35 }
36
```

Listing 11 逆序获取元素

14) 链表排序

链表排序主要采用插入排序的方法，减少链表的遍历次数和操作次数，具体算法如图1-3所示。

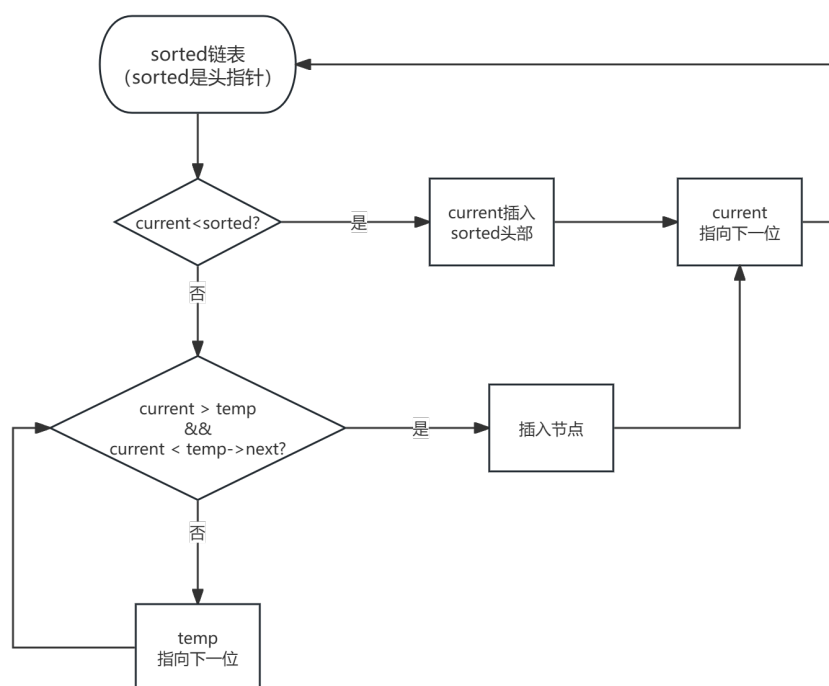


图 1-3 链表插入排序流程图

代码实现如下：

```

1 status SortList(LinkList *L)
2 {
3     if(*L == NULL || (*L)->next == NULL)
4         return INFEASIBLE;
5
6     LNode *current = (*L)->next;
7     LNode *next = NULL;
8     LNode *sorted = NULL;
9
10    while(current != NULL)
11    {
12        next = current->next; // 保存下一个节点
13        if(sorted == NULL || sorted->data >= current->data) // 插
14        入到头部
15        {
16            current->next = sorted;
17            sorted = current;
18        }
19        else // 插入到中间或尾部
20        {
21            LNode *temp = sorted;

```

```
21         while(temp->next != NULL && temp->next->data <
current->data)
22         {
23             temp = temp->next;
24         }
25         current->next = temp->next;
26         temp->next = current;
27     }
28     current = next; // 继续处理下一个节点
29 }
30
31 (*L)->next = sorted; // 更新头指针
32
33 return OK;
34 }
```

Listing 12 链表排序

15) 多链表操作

在本代码的实现中，多链表操作实质上是对一个多链表数组进行了基本的增删改查操作。多链表的管理主要包括添加链表、删除链表、切换链表、定位链表和输出链表清单等功能。代码实现并不复杂，主要是对多链表构成的顺序表进行操作，具体代码见附录。

1.4 系统测试

下面是参考测试用例进行的系统测试。

测试步骤 1：链表初始化

测试描述：测试链表初始化功能是否正常工作

```
请选择你的操作 [0~16]:15
输入表名:表一
插入成功!
```

预期结果

```
-----
Choose your operations [0~21]:18
Please input the name of the list you want to add: list1
Add the list list1, list id: 1
AddList successfully!
Press any key to continue...
```

实际结果

图 1-4 链表初始化测试结果对比

测试步骤 2：插入元素

测试描述：测试链表销毁功能是否正常工作

```
请选择你的操作 [0~16]:10
你想操作第几个线性表:
1
输入你想插入的元素:
6
输入插入位置:
1
插入成功!
```

预期结果

```
-----
Choose your operations [0~21]:10
Please input the index and the element you want to insert: 2 7
ListInsert successfully!
Press any key to continue...
```

实际结果

图 1-5 链表插入测试结果对比

测试步骤 3：链表判空

测试描述：测试链表判空功能是否正常工作

```
请选择你的操作 [0~16]:4
你想操作第几个线性表:
1
线性表不为空!
```

预期结果

```
-----
Choose your operations [0~21]:6
Please input the index of the element you want to get: 2
The element at index 2 is 7
Press any key to continue...
```

实际结果

图 1-6 链表判空测试结果对比

测试步骤 4：链表长度

测试描述：测试链表长度功能是否正常工作

```
请选择你的操作 [0~16]:5
你想操作第几个线性表:
1
线性表的长度是3
```

预期结果

```
-----
0. Return to Main Menu
-----
Choose your operations [0~21]:5
The length of the list is 3
Press any key to continue...
```

实际结果

图 1-7 链表长度测试结果对比

测试步骤 5：链表元素获取

测试描述：测试链表元素获取功能是否正常工作

```
你想操作第几个线性表:
1
输入你要查找的元素的位置:2
第2个元素是7
```

预期结果

```
Choose your operations [0~21]:6
Please input the index of the element you want to get: 2
The element at index 2 is 7
Press any key to continue...
```

实际结果

图 1-8 链表元素获取结果对比

测试步骤 6：元素前驱

测试描述：测试链表元素前驱功能是否正常工作

```
你想操作第几个线性表:
1
输入你想寻找前驱的元素:
6
这个元素没有前驱!_
```

预期结果 1

```
你想操作第几个线性表:
1
输入你想寻找前驱的元素:
6
前驱元素是6
```

预期结果 2

```
-----
Choose your operations [0~21]:8
Please input the element you want to find the prior of: 6
PriorElem failed!
Press any key to continue...|
```

实际结果 1

```
-----
Choose your operations [0~21]:8
Please input the element you want to find the prior of: 7
The prior element of 6 is found!
Press any key to continue...
```

实际结果 2

图 1-9 链表元素前驱测试结果对比

测试步骤 7：元素后继

测试描述：测试链表元素后继功能是否正常工作

```
你想操作第几个线性表：
1
输入你想寻找后继的元素：
3
后继元素是7
```

预期结果 1

```
Choose your operations [0~21]:9
Please input the element you want to find the next of: 8
NextElem failed!
Press any key to continue...
```

实际结果 1

```
你想操作第几个线性表：
1
输入你想寻找后继的元素：
3
这个元素没有后继!_
```

预期结果 2

```
Choose your operations [0~21]:9
Please input the element you want to find the next of: 6
The next element of 7 is found!
Press any key to continue...|
```

实际结果 2

图 1-10 链表元素后继测试结果对比

测试步骤 8：链表遍历

测试描述：测试链表遍历功能是否正常工作

```
表名      :   表一
元素      :   6 7 8
```

预期结果

```
Choose your operations [0~21]:12
6 7 8
ListTraverse successfully!
Press any key to continue...|
```

实际结果

图 1-11 链表遍历测试结果对比

测试步骤 9：链表读写

测试描述：测试链表读写功能是否正常工作

```
你想操作第几个线性表：
1
保存成功!
```

预期结果 1

```
Choose your operations [0~21]:13
Please input the file name to save the list: list_test.out
SaveList successfully!
Press any key to continue...|
```

实际结果 1

```
给这个表起个名字:读取
载入成功!
```

预期结果 2

```
Choose your operations [0~21]:14
Please input the file name to load the list: list_test.out
Lists already exist, rewrite it? (y/n): y
LoadList successfully!
Press any key to continue...
```

实际结果 2

图 1-12 链表读写测试结果对比

测试步骤 10：删除元素

测试描述：测试链表元素删除功能是否正常工作

```
Choose your operations [0~21]:11
Please input the index of the element you want to delete: 2
ListDelete successfully! Deleted element: 7
Press any key to continue...
```

实际结果 1

```
-----
Choose your operations [0~21]:12
6 8
ListTraverse successfully!
Press any key to continue...
```

实际结果 2

图 1-13 链表元素删除测试结果对比

测试步骤 11：多链表操作

测试描述：测试多链表操作功能是否正常工作

```
Choose your operations [0~21]:19
Please input the name of the list you want to remove: list1
RemoveList successfully!
Press any key to continue...|
```

实际结果 1

```
Choose your operations [0~21]:22
List 1: list2
Press any key to continue...|
```

实际结果 2

图 1-14 多链表操作测试结果对比

测试描述：测试多链表添加功能是否正常工作

```
-----
请选择你的操作 [0~16]:15
输入表名:新表
插入成功!_
```

预期结果

```
Choose your operations [0~21]:18
Please input the name of the list you want to add: list3
Add the list list3, list id: 2
AddList successfully!
Press any key to continue...|
```

实际结果

图 1-15 多链表操作测试结果对比

测试步骤 12：多链表定位

测试描述：测试多链表定位功能是否正常工作

```
请选择你的操作 [0~16]:17
输入表名:读取
位置在1
```

预期结果

```
Choose your operations [0~21]:21
Please input the name of the list you want to locate: list3
LocateList successfully! List id: 2
Press any key to continue...|
```

实际结果

图 1-16 多链表操作测试结果对比

测试步骤 13：链表反转

测试描述：测试链表反转功能是否正常工作

1、链表反转结果为 4, -5, 1, 2, -1, 4, -3, 1, -2
2、删除链表倒数第 2 个节点结果为 4, -5, 1, 2, -1, 4, -3, -2

预期结果

```
Choose your operations [0~21]:12
4 -5 1 2 -1 4 -3 1 -2
ListTraverse successfully!
Press any key to continue...|
```

实际结果

图 1-17 链表反转测试结果对比

测试步骤 14：链表排序

测试描述：测试链表排序功能是否正常工作

```
Choose your operations [0~21]:12
4 -5 1 2 -1 4 -3 1 -2
ListTraverse successfully!
Press any key to continue...|
```

初始状态

```
Choose your operations [0~21]:12
-5 -3 -2 -1 1 1 2 4 4
ListTraverse successfully!
Press any key to continue...|
```

实际结果

图 1-18 链表排序测试结果对比

测试步骤 15：逆序删除元素

测试描述：测试链表逆序删除元素功能是否正常工作

1、链表反转结果为 4, -5, 1, 2, -1, 4, -3, 1, -2
2、删除链表倒数第 2 个节点结果为 4, -5, 1, 2, -1, 4, -3, -2

预期结果

```
4 -5 1 2 -1 4 -3 -2
ListTraverse successfully!
Press any key to continue...|
```

实际结果

图 1-19 链表逆序删除测试结果对比

1.5 实验小结

通过本次基于链式存储结构的线性表实现实验，我深入理解了链式存储结构的原理和应用。与顺序存储结构相比，链式存储结构在动态内存分配和管理方面具有显著优势，特别适合于频繁进行插入和删除操作的场景。

在实验过程中，我实现了包括链表初始化、插入、删除、查找等基本操作，以及链表反转、排序和逆序删除等高级功能。这些操作的实现帮助我牢固掌握了指针操作和内存管理的技术要点。特别是在实现链表反转和排序功能时，需要巧妙地处理指针关系，这对我理解数据结构的实际应用有很大帮助。

多链表管理系统的设计与实现是本实验的亮点，通过使用指针数组和状态机模式，实现了多链表的并发管理。这种设计模式不仅使代码结构更加清晰，也提高了系统的可扩展性和维护性。同时，全局状态维护和切换链表的设计思路对我未来开发更复杂系统有很好的指导意义。

在实验过程中，我也遇到了一些挑战。例如，在实现链表的逆序删除操作时，最初使用了链表反转的方法，后来通过快慢指针算法进行了优化，减少了操作步骤，提高了算法效率。这个过程让我更深入地理解了算法优化的重要性。

总的来说，这次实验不仅帮助我巩固了链表相关的理论知识，也提升了我的编程实践能力。通过亲自实现各种链表操作，我对数据结构的理解更加立体和深入。

2 基于邻接表的图实现

邻接表是图的一种存储结构，适用于稀疏图的存储。它使用链表来存储每个顶点的邻接顶点，能够有效地节省空间。本次实验主要实现了基于邻接表的图的基本操作，通过一个集成的系统封装了相关操作，并提供了一个简单的用户界面。

2.1 问题描述

本实验主要解决以下几个问题：

- 图的邻接表的基本数据结构和设计
- 图的邻接表的创建、销毁、清空等基本操作
- 邻接表的进阶操作功能以及邻接表的文件输入输出
- 邻接表操作系统的构筑以及多邻接表显示管理

下面从系统总体设计构筑以及系统具体功能实现两个方面来解决上述问题

2.2 系统设计

2.2.1 数据结构设计

本实验涉及的数据结构主要有如下几种：头节点，表节点，邻接表，多邻接表等。下面分别介绍这些数据结构的定义和作用。

1) 表节点

表节点或称为弧节点，是邻接表的基本组成部分。它包含了指向下一个弧节点的指针和弧的终点信息。表节点的定义如下：

```
1 typedef struct ArcNode {           //表结点类型定义
2     int adjvex;                    //顶点位置编号
3     struct ArcNode *nextarc;      //下一个表结点指针
4 } ArcNode;
5
```

Listing 13 表节点定义

2) 头节点

头节点是邻接表节点的入口，它包含了当前节点的信息和该节点第一条弧的指针。头节点的定义如下：

```
1 typedef struct VNode{           //头结点及其数组类型定义
2     VertexType data;           //顶点信息
3     ArcNode *firstarc;         //指向第一条弧
4 } VNode, AdjList[MAX_VERTEX_NUM];
5
```

Listing 14 头节点定义

其中，*VertexType* 是顶点信息的类型，在本实验中是一个包含了关键值 *key* 以及名称 *name* 的结构体。

3) 邻接表

邻接表结构是对节点结构的集合，包含了一个头节点数组和当前图的顶点数以及弧数。邻接表的定义如下：

```
1 typedef struct { //邻接表的类型定义
2     AdjList vertices; //头结点数组
3     int vexnum, arcnum; //顶点数、弧数
4     GraphKind kind; //图的类型
5 } ALGraph;
6
```

Listing 15 邻接表定义

4) 多邻接表

多邻接表本质上是一个邻接表数组，包含了邻接表以及邻接表对应的名称和索引等信息。多邻接表的定义如下：

```
1 typedef struct {
2     ALGraph Ggroup[MAX_VERTEX_NUM];
3     int Gnum;
4     char name[MAX_VERTEX_NUM][20];
5 } GraphGroup;
6
```

Listing 16 多邻接表定义

系统功能设计 用户可以通过系统进行的操作可以分为两个方面：邻接表的基本操作以及多邻接表的管理。具体设计架构如下：

5) 单链表操作状态机

单邻接表操作状态机主要包括以下几个状态：创建、销毁、插入、删除、查找、显示等。每个状态对应一个函数，用户可以通过输入指令来切换状态并执行相应的操作。具体来说，程序进入状态机后，将进入一个循环，等待用户输入命令，根据输入的命令执行相应的操作。每个操作完成后，系统会返回到主菜单状态，等待下一次用户输入，直到用户输入退出指令。具体结构如下：

```
1  while(op)
2  {
3      /*
4      Operating Menu
5      */
6      scanf("%d", &op);           // 获取用户输入的操作码
7      if(op == 0)
8      {
9          printf("Returning to Main Menu...\n");
10         op = 1;
11         break;                   // 如果用户输入0，则返回
主菜单
12     }
13     switch(op)
14     {
15         case 1:
16         case 2:
17             ...
18     }
19
20     printf("Press any key to continue...");
21     getchar(); // 清除缓冲区中的换行符
22     getchar(); // 等待用户输入
23 }
24
25 op = 1; // 重置操作码，返回主菜单
26
```

Listing 17 单邻接表操作状态机

6) 多邻接表管理状态机

多邻接表管理状态机的核心在于如何实现对任意一个链表的创建，销毁以及实现所有单链表的具体操作，同时要在邻接表切换之后能够保留之前的操作状态。此外，为了减少代码的复杂度，实现单邻接表操作和多邻接表操作的代码复用，系统采用了一个指针管理的并发模式，每次进入单邻接表管理部分之前会读取当前待操作邻接表 `G_operated` 对应的实际链表指针，实际操作过程中只使用 `G_operated` 作为函数入口的操作对象。具体结构如下：


```

1      if(mode == 2)
2      {
3          system("cls");    printf("\n\n");
4          printf("Multi-Graphs Operation Mode:\n");
5          if(graph_idx >= Graphs.Gnum)           //判断当前索引是否指向一个有效的邻接表
6          {
7              G_operated = NULL;
8              printf("No graph exists!\n");
9          }
10         else
11         {
12             G_operated = &Graphs.Ggroup[graph_idx];
13             printf("Current list: %s", Graphs.name[graph_idx
14         ]);
15             printf("Graph id: %d", graph_idx + 1);
16             if(Graphs.Ggroup[graph_idx].vexnum == 0) //判断当前邻接表是否为空
17                 printf("(Empty)");
18         }
19     }

```

Listing 18 多邻接表管理状态机

如上述代码所示，系统首先判断当前是否处于多邻接表管理模式，如果是，则获取当前待操作邻接表的指针，具体来说是通过获取 main 函数中的全局变量 **graph_idx** 来获取当前待操作邻接表的索引，如果要切换邻接表，只需修改 **graph_idx** 即可。此外，系统还会在每次进入多邻接表管理模式时输出当前待操作邻接表的名称和索引，并判断当前是否有可操作的邻接表，可操作的邻接表是否为空，如果不是，则输出提示信息。

7) 最终效果

最终效果如图：2-1所示，用户可以通过输入指令来进行邻接表的创建、销毁、插入、删除、查找等操作，同时可以通过多链表管理状态机来管理多个链式线性表。

```
Multi-BiTrees Operation Mode:
-----
No graph exists!
-----
Menu for Operation Selection
-----
1. CreateGraph      7. InsertVex
2. DestroyGraph    8. DeleteVex
3. GetLocation     9. InsertArc
4. PutVex          10. DeleteArc
5. FirstAdjVex     11. DFSTraverse
6. NextAdjVex      12. BFSTraverse
-----
File Operation Menu
-----
13. SaveGraph      14. LoadGraph
-----
Additional Functions Menu
-----
15. VerticesSetLessThanK  17. ConnectedComponentsNum
16. ShortestPath          18. PrintGraphList
-----
Multi-BiTrees Operation Selection
-----
19. AddGraph      22. PrintGraphGroup
20. RemoveGraph   23. SwitchGraph
21. LocateGraph   24. InitGraphGroup
-----
0. Exit
-----
```

图 2-1 链式线性表操作系统界面

2.3 系统实现

完成系统架构介绍后，下文将介绍系统的具体实现，包括各个主要函数的实现思想，复杂函数可辅助流程图进行说明，函数和系统实现的源代码放在附录中。

本系统实现了以下功能：

- | | | |
|-------------|----------|------------|
| • 邻接表初始化 | 的下一邻接点 | • 广度优先搜索 |
| • 邻接表销毁 | • 插入节点 | • 查找指定距离节点 |
| • 获取节点位置 | • 删除节点 | • 最短路径 |
| • 节点赋值 | • 插入弧 | • 获取连通分量 |
| • 获取节点第一邻接点 | • 删除弧 | • 邻接表读写 |
| • 获取节点指定邻接点 | • 深度优先搜索 | • 多邻接表操作 |

下面具体介绍各个功能的实现方式

1) 邻接表初始化

邻接表初始化函数主要用于创建一个新的链表，并返回创建状态。具体实现如下：

```

1 status CreateCraph(ALGraph &G,VertexType V[],KeyType VR[][2])
2 {
3     int i,j;
4     ArcNode *p1, *p2;
5
6     for(i = 0; V[i].key != -1; i++);
7
8     G.vexnum = i;
9     G.arcnum = 0;
10
11     // 节点赋值
12     for(i = 0; i < G.vexnum; i++)
13     {
14         G.vertices[i].data = V[i];
15         G.vertices[i].firstarc = NULL;
16     }
17
18     if(i < G.vexnum || i == 0)
19         return ERROR;
20
21     // 检查关键字是否重复
22     for(i = 0; i < G.vexnum; i++)
23     {
24         for(j = i + 1; j < G.vexnum; j++)

```

```
25     {
26         if(V[i].key == V[j].key)
27             return ERROR;
28     }
29 }
30
31 //对每一条弧，向弧对应的顶点的邻接表中插入弧节点
32 for(j = 0; VR[j][0] != -1; j++)
33 {
34     int loc1, loc2;
35     loc1 = GetLocation(G, VR[j][0]);
36     loc2 = GetLocation(G, VR[j][1]);
37
38     if(loc1 == -1 || loc2 == -1)
39         return ERROR;
40
41     p1 = (ArcNode *)malloc(sizeof(ArcNode));
42     p1 -> adjvex = loc2;
43     p2 = (ArcNode *)malloc(sizeof(ArcNode));
44     p2 -> adjvex = loc1;
45
46     //采用头插法将弧节点插入到邻接表中
47     ArcNode *temp = G.vertices[loc1].firstarc;
48     G.vertices[loc1].firstarc = p1;
49     p1 -> nextarc = temp;
50     temp = G.vertices[loc2].firstarc;
51     G.vertices[loc2].firstarc = p2;
52     p2 -> nextarc = temp;
53     G.arcnum++;
54
55 }
56
57 G.kind = UDN;
58 return OK;
59 }
60
```

Listing 19 邻接表初始化

2) 邻接表销毁

邻接表销毁只需要按照节点顺序遍历节点，并按照没一个节点将节点的弧链表清空即可，代码见附录

3) 邻接表获取节点位置

邻接表获取节点位置函数主要用于获取指定关键字的节点在邻接表中的位置，只需遍历邻接表的头节点数组并比较关键字即可，代码见附录

4) 头节点赋值

头节点赋值函数主要用于将指定的顶点信息赋值给指定位置的头节点，只需获取位置并复制即可，代码见附录

5) 获取头节点第一邻接点

获取第一邻接点只需要获取指定头节点索引并访问其弧链表的第一个节点即可，代码见附录

6) 获取指定邻接点的下一邻接点

获取指定邻接点的下一邻接点函数主要用于获取指定头节点索引和弧节点索引的下一弧节点，需要遍历表头结点获得索引并遍历其弧链表获取下一弧节点，代码如下：

```
1 int NextAdjVex(ALGraph G,KeyType v,KeyType w)
2 //v对应G的一个顶点,w对应v的邻接顶点;操作结果是返回v的(相对于w)
   下一个邻接顶点的位序;如果w是最后一个邻接顶点,或v、w对应顶点
   不存在,则返回-1。
3 {
4     int i = GetLocation(G, v);           // 获取指定头节点索引
5     int j = GetLocation(G, w);           // 获取指定弧节点对应终点
   的索引
6     if(i == -1) return -1;
7
8     ArcNode *p = G.vertices[i].firstarc;
9     while(p != NULL)
10    {
11        if(p -> adjvex == j)
12            break;
13        p = p -> nextarc;                 // 遍历弧链表,找到指定弧
   节点
14    }
15
16    if(p == NULL || p -> nextarc == NULL) return -1;
17
18    return p -> nextarc -> adjvex;
19 }
```

Listing 20 获取指定邻接点的下一邻接点

该函数首先获取指定头节点索引，然后遍历其弧链表，找到指定弧节点索引的弧节点，并返回其下一弧节点的索引，如果不存在则返回-1。

7) 插入头节点

插入头节点函数主要用于在邻接表中插入一个新的头节点，只需将新节点插入到头节点数组中即可。注意要检查头节点数组是否已满以及新节点的关键

字是否重复，代码实现如下：

```
1 status InsertVex(ALGraph &G, VertexType v)
2 //在图G中插入顶点v，成功返回OK，否则返回ERROR
3 {
4     if(G.vexnum >= MAX_VERTEX_NUM) return ERROR;
5
6     for(int i = 0; i < G.vexnum; i++)
7     {
8         if(G.vertices[i].data.key == v.key)
9             return ERROR;
10    } // 检查关键字是否重复
11
12    G.vertices[G.vexnum].data = v;
13    G.vertices[G.vexnum].firstarc = NULL;
14    G.vexnum++;
15
16    return OK;
17 }
```

Listing 21 插入头节点

8) 删除头节点

删除头节点函数主要用于在邻接表中删除一个头节点，并清空其弧链表。注意不仅要删除头节点的弧链表，还要删除头节点邻接点的弧链表中对应的弧，代码实现如下：

```
1 status DeleteVex(ALGraph &G, KeyType u)
2 {
3     int i = GetLocation(G, u);
4     if(i == -1) return ERROR;
5
6     // 计数待删除的边数
7     int edgeCount = 0;
8
9     // 释放顶点的出边
10    ArcNode *p = G.vertices[i].firstarc;
11    while(p != NULL)
12    {
13        ArcNode *temp = p;
14        p = p->nextarc;
15        free(temp);
16        edgeCount++; // 每释放一条边，计数加1
17    }
18
19    // 处理其他顶点的入边和对应的出边
20    for(int j = 0; j < G.vexnum; j++)
21    {
22        if(j == i) continue; // 跳过被删除的顶点
23
24        ArcNode *p1 = G.vertices[j].firstarc;
```

```

25     ArcNode *p2 = NULL;
26
27     while(p1 != NULL)
28     {
29         if(p1->adjvex == i) // 找到指向要删除顶点的边
30         {
31             // 删除这条边
32             if(p2 == NULL) // 如果是第一条边
33             {
34                 G.vertices[j].firstarc = p1->nextarc;
35                 free(p1);
36                 p1 = G.vertices[j].firstarc;
37             }
38             else // 如果不是第一条边
39             {
40                 p2->nextarc = p1->nextarc;
41                 free(p1);
42                 p1 = p2->nextarc;
43             }
44             edgeCount++; // 每删除一条边，计数加1
45         }
46         else
47         {
48             // 调整大于i的adjvex值
49             if(p1->adjvex > i)
50                 p1->adjvex--;
51
52             p2 = p1;
53             p1 = p1->nextarc;
54         }
55     }
56 }
57
58 // 移动顶点数组
59 for(int j = i; j < G.vexnum - 1; j++)
60 {
61     G.vertices[j] = G.vertices[j + 1];
62 }
63
64 // 更新顶点数和边数
65 G.vexnum--;
66 G.arcnum -= edgeCount / 2; // 因为是无向图，每条边计算了两次
67
68 if(G.vexnum == 0) return ERROR;
69
70 // printf("Finished\n");
71
72 return OK;
73 }

```

Listing 22 删除头节点

该函数首先获取指定头节点索引，然后遍历其弧链表，释放所有弧节点，并将其

他头节点的弧链表中对应的弧节点删除，最后将头节点数组中的节点向前移动一位，并更新顶点数和弧数。此外，注意邻接表的定义，还要删除邻接表的节点数目和弧数目，这里采用了握手定理的方式来计算弧数目，即每条边被计算了两次，因此需要除以 2。

9) 插入弧

插入弧函数主要用于在邻接表中插入一条新的弧，注意要检查弧的起点和终点是否存在，同时要注意需要在弧的两个对应顶点的邻接表中都插入弧节点。代码实现如下：

```
1 status InsertArc(ALGraph &G,KeyType u,KeyType v)
2 {
3     //获取起点和终点的索引
4     int loc1 = GetLocation(G, u);
5     int loc2 = GetLocation(G, v);
6     if(loc1 == -1 || loc2 == -1) return ERROR;
7
8     //检查弧是否已经存在
9     ArcNode *p = G.vertices[loc1].firstarc;
10    while(p != NULL)
11    {
12        if(p->adjvex == loc2) return ERROR;
13        p = p->nextarc;
14    }
15
16    //赋值
17    ArcNode *p1 = (ArcNode *)malloc(sizeof(ArcNode));
18    p1 -> adjvex = loc2;
19    p1 -> nextarc = G.vertices[loc1].firstarc;
20    G.vertices[loc1].firstarc = p1;
21
22    ArcNode *p2 = (ArcNode *)malloc(sizeof(ArcNode));
23    p2 -> adjvex = loc1;
24    p2 -> nextarc = G.vertices[loc2].firstarc;
25    G.vertices[loc2].firstarc = p2;
26
27    G.arcnum++;
28
29    return OK;
30 }
```

Listing 23 插入弧

该函数首先获取指定弧的起点和终点的索引，然后检查弧是否已经存在，如果不存在，则在起点和终点的邻接表中分别插入弧节点，并更新弧数。

10) 删除弧

删除弧函数主要用于在邻接表中删除一条弧，注意要检查弧的起点和终点是否存在，同时要注意需要在弧的两个对应顶点的邻接表中都删除弧节点，实现与插入弧类似，只需遍历弧链表找到对应的弧节点并删除即可。代码见附录

11) 深度优先搜索

深度优先搜索（DFS）是一种遍历图的算法，主要用于访问图中的所有顶点。该算法从一个起始顶点开始，沿着一条路径深入到尽可能远的顶点，然后回溯到上一个顶点继续搜索。在邻接表中实现深度优先搜索时，可以使用递归或栈来实现。本次实验的 DFS 采用了两层函数封装，外层函数用于提供深度优先搜索操作函数的接口，内层函数提供具体的递归搜索，代码实现如下：

```
1 // 辅助函数：递归进行深度优先搜索
2 void DFS(ALGraph &G, int v, int visited[], void (*visit)(
  VertexType)) {
3 // 访问当前顶点
4 if(visit != NULL)
5     visit(G.vertices[v].data);
6     visited[v] = true;
7
8 // 遍历当前顶点的所有邻接顶点
9     ArcNode *p = G.vertices[v].firstarc;
10    while (p != NULL) {
11        // 如果邻接顶点未被访问，则递归访问它
12        if (!visited[p->adjvex]) {
13            DFS(G, p->adjvex, visited, visit);
14        }
15        p = p->nextarc;
16    }
17 }
18
19 status DFSTraverse(ALGraph &G, void (*visit)(VertexType))
20 //对图G进行深度优先搜索遍历，依次对图中的每一个顶点使用函数visit
  访问一次，且仅访问一次
21 {
22     if (G.vexnum == 0) return ERROR; // 如果图为空，返回错误
23
24     // 初始化访问标记数组
25     int visited[MAX_VERTEX_NUM];
26     for (int i = 0; i < G.vexnum; i++) {
27         visited[i] = false;
28     }
29
30     // 从每个未访问的顶点开始进行深度优先搜索
31     for (int i = 0; i < G.vexnum; i++) {
32         if (!visited[i]) {
33             DFS(G, i, visited, visit);
```

```

34     }
35 }
36 return OK;
37 }

```

Listing 24 深度优先搜索

该函数首先初始化访问标记数组，然后从每个未访问的顶点开始进行深度优先搜索，访问过程中调用用户提供的访问函数。

12) 广度优先搜索

广度优先搜索（BFS）是一种遍历图的算法，主要用于访问图中的所有顶点。该算法从一个起始顶点开始，首先访问该顶点，然后访问所有与该顶点直接相连的顶点，再依次访问这些顶点的邻接顶点。在邻接表中实现广度优先搜索时，可以使用队列来实现。代码实现如下：

```

1 status BFSTraverse(ALGraph &G, void (*visit)(VertexType))
2 //对图G进行广度优先搜索遍历，依次对图中的每一个顶点使用函数visit
  访问一次，且仅访问一次
3 {
4     if (G.vexnum == 0) return ERROR; // 如果图为空，返回错误
5
6     // 初始化访问标记数组
7     bool visited[MAX_VERTEX_NUM];
8     for (int i = 0; i < G.vexnum; i++) {
9         visited[i] = false;
10    }
11
12    // 创建队列用于BFS
13    int queue[MAX_VERTEX_NUM];
14    int front = 0;
15    int rear = 0;
16
17    // 从每个未访问的顶点开始进行广度优先搜索
18    for (int i = 0; i < G.vexnum; i++) {
19        if (!visited[i]) {
20            // 访问当前顶点
21            visit(G.vertices[i].data);
22            visited[i] = true;
23
24            // 将当前顶点入队
25            queue[rear++] = i;
26
27            // 队列非空时循环处理
28            while (front < rear) {
29                // 出队一个顶点
30                int v = queue[front++];
31
32                // 访问该顶点的所有未访问邻接点
33                ArcNode *p = G.vertices[v].firstarc;

```

```
34         while (p != NULL) {
35             int w = p->adjvex;
36             if (!visited[w]) {
37                 visit(G.vertices[w].data);
38                 visited[w] = true;
39                 queue[rear++] = w; // 新发现的顶点入队
40             }
41             p = p->nextarc;
42         }
43     }
44 }
45 }
46 return OK;
47 }
```

Listing 25 广度优先搜索

该函数首先初始化访问标记数组和队列，然后从每个未访问的顶点开始进行广度优先搜索，访问过程中调用用户提供的访问函数。

13) 查找距离小于k的顶点

查找距离小于k的顶点在本质上是进行了广度优先搜索，即进行深度为k的广度优先搜索，并返回第k层及之前的搜索的结果。代码实现上是对广度优先搜索队列进行了一定的改造，具体代码如下：

```
1 VertexType *VerticesSetLessThanK(ALGraph G, KeyType v, int k)
2 {
3     int i;
4     int visited[MAX_VERTEX_NUM] = {0};
5     int distance[MAX_VERTEX_NUM];
6     VertexType *result = (VertexType *)malloc(G.vexnum * sizeof(
VertexType));
7     int count = 0;
8
9     for(int i = 0; i < G.vexnum; i++)
10     {
11         result[i].key = -1; // 初始化结果数组
12     }
13
14     // 初始化访问标记和距离数组
15     for (i = 0; i < G.vexnum; i++) {
16         visited[i] = 0;
17         distance[i] = -1;
18     }
19
20     // 查找起始顶点的位置
21     int startIndex = GetLocation(G, v);
22     if (startIndex == -1) {
23         free(result);
24         return NULL; // 起始顶点不存在
25     }
26
27     // 创建队列用于BFS
28     int queue[MAX_VERTEX_NUM];
29     int front = 0, rear = 0;
30
31     // 将起始顶点入队
32     queue[rear++] = startIndex;
33     visited[startIndex] = 1;
34     distance[startIndex] = 0;
35     result[count++] = G.vertices[startIndex].data;
36
37     // BFS遍历
38     while (front < rear) {
39         // 出队一个顶点
40         int current = queue[front++];
41
42         // 如果当前顶点的距离已经等于k-1，那么它的邻接点距离将是k
, 不需要继续遍历
43         if (distance[current] == k - 1) {
44             continue;
```

```

45     }
46
47     // 访问所有邻接点
48     ArcNode *p = G.vertices[current].firstarc;
49     while (p != NULL) {
50         int adjIndex = p->adjvex;
51         if (!visited[adjIndex]) {
52             visited[adjIndex] = 1;
53             distance[adjIndex] = distance[current] + 1;
54
55             // 添加到结果集
56             result[count++] = G.vertices[adjIndex].data;
57
58             // 只有当距离小于k-1的顶点才入队，以便继续遍历其
邻接点
59             if (distance[adjIndex] < k - 1) {
60                 queue[rear++] = adjIndex;
61             }
62         }
63         p = p->nextarc;
64     }
65 }
66
67 // 返回结果数组的一个副本，只包含需要的部分
68 VertexType *finalResult = (VertexType *)malloc((count + 1) *
sizeof(VertexType));
69 for (i = 0; i < count; i++) {
70     finalResult[i] = result[i];
71 }
72 finalResult[count].key = -1; // 设置结束标记
73 free(result);
74
75 return finalResult;
76 }

```

Listing 26 查找距离小于 k 的顶点

该函数首先初始化访问标记和距离数组，然后从起始顶点开始进行广度优先搜索，访问过程中将距离小于 k 的顶点添加到结果数组中，并返回结果数组。

14) 最短路径

获取最短路径函数主要用于获取两个顶点之间的最短路径，在无权图中可以令每一条边的权为 1，不存在负权故可以考虑 *Dijkstra* 算法。该算法通过维护一个距离数组和前驱数组来记录每个顶点到起始顶点的最短距离和前驱顶点。算法流程图如下所示：代码实现如下：

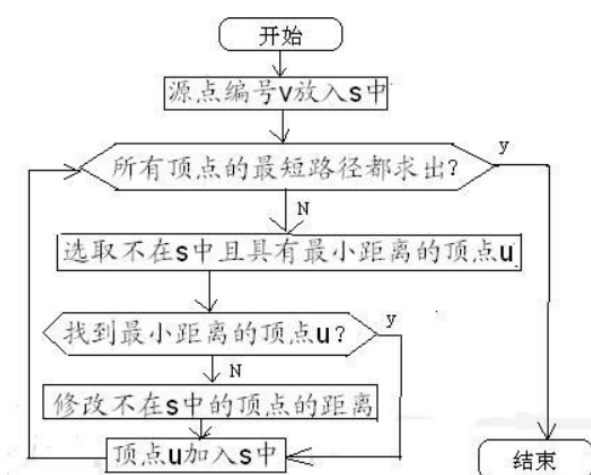


图 2-2 Dijkstra 算法流程图

```

1  //Dijkstra algorithm
2  int ShortestPath(ALGraph G, KeyType u, KeyType v)
3  {
4      int i, j, min, minIndex;
5      int visited[MAX_VERTEX_NUM] = {0};
6      int dist[MAX_VERTEX_NUM];
7
8      int startIndex = GetLocation(G, u);
9      int endIndex = GetLocation(G, v);
10     if(startIndex == -1 || endIndex == -1) return -1; // 起始或结束顶点不存在
11
12     if(startIndex == endIndex) return 0; // 起始和结束顶点相同
13
14     for(i = 0; i < G.vexnum; i++)
15     {
16         dist[i] = 32767;
17     }
18

```

```

19     dist[startIndex] = 0; // 起始顶点到自身的距离为0
20
21     for(i = 0; i < G.vexnum; i++)
22     {
23         min = 32767;
24         minIndex = -1;
25
26         for(j = 0; j < G.vexnum; j++)
27         {
28             if(!visited[j] && dist[j] < min)
29             {
30                 min = dist[j];
31                 minIndex = j;
32             }
33         }
34
35         if(minIndex == -1) break; // 所有可达顶点都已访问
36
37         visited[minIndex] = 1; // 标记当前顶点为已访问
38
39         if(minIndex == endIndex) return dist[minIndex]; // 找到目
标顶点
40
41         ArcNode *p = G.vertices[minIndex].firstarc;
42         while(p != NULL)
43         {
44             int adjIndex = p -> adjvex;
45
46             int weight = 1;
47
48             if(!visited[adjIndex] && dist[minIndex] != 32767 &&
49                 dist[minIndex] + weight < dist[adjIndex])
50             {
51                 dist[adjIndex] = dist[minIndex] + weight;
52             }
53
54             p = p -> nextarc;
55         }
56     }
57
58     if(dist[endIndex] == 32767)
59     {
60         return -1; // 目标顶点不可达
61     }
62
63     return dist[endIndex]; // 返回最短路径长度
64 }
65

```

Listing 27 最短路径

该函数首先初始化访问标记和距离数组，然后从起始顶点开始进行 *Dijkstra* 算法，访问过程中更新距离数组，直到找到目标顶点或所有可达顶点都已访问。*Dijkstra*

算法具体实现中，采用了 *visited* 数组来标记已访问的顶点，*dist* 数组来记录每个顶点到起始顶点的最短距离，最后返回目标顶点的最短距离。

15) 图的连通分支

获取图的连通分支函数主要用于获取图的所有连通分支，可以通过深度优先搜索或广度优先搜索来实现。该函数遍历图中的所有顶点，若未被访问，则进行一次深度优先搜索或广度优先搜索，直到所有顶点都被访问。代码实现如下：

```
1  int ConnectedComponentsNum(ALGraph G)
2  {
3      int i, j, count = 0;
4      int visited[MAX_VERTEX_NUM] = {0};
5
6      for(i = 0; i < G.vexnum; i++)
7      {
8          if(!visited[i])
9          {
10             count++;
11             // 深度优先搜索遍历当前连通分量
12             DFS(G, i, visited, NULL);
13         }
14     }
15
16     return count;
17 }
```

Listing 28 图的连通分支

该函数首先初始化访问标记数组，然后遍历图中的所有顶点，若未被访问，则进行一次深度优先搜索，直到所有顶点都被访问，最后返回连通分支的数量。

16) 邻接表读写

邻接表的读写函数主要用于将邻接表保存到文件中或从文件中读取邻接表。该函数首先打开文件，然后按照邻接表的格式将顶点和弧写入文件或从文件中读取顶点和弧。输入输出的格式为：头节点 + 弧

```
1 status SaveGraph(ALGraph G, char FileName[])
2 //将图的数据写入到文件FileName中
3 {
4     FILE *fp;
5     int i, j, edgeCount = 0;
6     ArcNode *p;
7
8     // 创建边的数组用于排序
9     typedef struct {
10         KeyType v1;
11         KeyType v2;
12     } Edge;
13
14     Edge edges[MAX_ARC_NUM];
15
16     // 打开文件，如果失败则返回ERROR
17     if ((fp = fopen(FileName, "w")) == NULL) {
18         return ERROR;
19     }
20
21     // 写入图的基本信息：类型、顶点数和边数
22     fprintf(fp, "%d %d %d\n", G.kind, G.vexnum, G.arcnum);
23
24     // 写入所有顶点的信息（按照数组顺序）
25     for (i = 0; i < G.vexnum; i++) {
26         fprintf(fp, "%d %s\n", G.vertices[i].data.key, G.vertices[i].data.others);
27     }
28
29     // 收集所有边的信息
30     for (i = 0; i < G.vexnum; i++) {
31         p = G.vertices[i].firstarc;
32         while (p != NULL) {
33             // 对于无向图，只收集一次边（只有当i<p->adjvex时）
34             if (G.kind == DG || G.kind == DN || i < p->adjvex) {
35                 edges[edgeCount].v1 = G.vertices[i].data.key;
36                 edges[edgeCount].v2 = G.vertices[p->adjvex].data.
key;
37                 edgeCount++;
38             }
39             p = p->nextarc;
40         }
41     }
42
43     // 对边进行冒泡排序（先按v1排序，v1相同时按v2排序）
44     for (i = 0; i < edgeCount - 1; i++) {
```

```

45     for (j = 0; j < edgeCount - i - 1; j++) {
46         if (edges[j].v1 > edges[j + 1].v1 ||
47             (edges[j].v1 == edges[j + 1].v1 && edges[j].v2 >
edges[j + 1].v2)) {
48             Edge temp = edges[j];
49             edges[j] = edges[j + 1];
50             edges[j + 1] = temp;
51         }
52     }
53 }
54
55 // 写入排序后的边信息
56 for (i = 0; i < edgeCount; i++) {
57     fprintf(fp, "%d %d\n", edges[i].v1, edges[i].v2);
58 }
59
60 fclose(fp);
61 return OK;
62 }
63
64 status LoadGraph(ALGraph &G, char FileName[])
65 //读入文件FileName的图数据, 创建图的邻接表
66 {
67     FILE *fp;
68     int i, arcnum;
69     KeyType v1, v2;
70     VertexType vertices[MAX_VERTEX_NUM];
71     KeyType VR[MAX_ARC_NUM][2];
72     char others[20];
73
74     // 打开文件, 如果失败则返回ERROR
75     if ((fp = fopen(FileName, "r")) == NULL) {
76         return ERROR;
77     }
78
79     // 读取图的基本信息: 类型、顶点数和边数
80     if (fscanf(fp, "%d %d %d", &G.kind, &G.vexnum, &arcnum) != 3)
81     {
82         fclose(fp);
83         return ERROR;
84     }
85
86     // 读取顶点信息
87     for (i = 0; i < G.vexnum; i++) {
88         if (fscanf(fp, "%d %s", &vertices[i].key, others) != 2) {
89             fclose(fp);
90             return ERROR;
91         }
92         strcpy(vertices[i].others, others);
93     }
94
95     // 设置结束标记
96     vertices[i].key = -1;
97
98     // 读取边信息

```

```
98     i = 0;
99     while (fscanf(fp, "%d %d", &v1, &v2) == 2 && i < MAX_ARC_NUM)
100     {
101         VR[i][0] = v1;
102         VR[i][1] = v2;
103         i++;
104     }
105     // 设置边列表的结束标记
106     VR[i][0] = -1;
107     VR[i][1] = -1;
108
109     fclose(fp);
110
111     // 使用CreateGraph函数创建图
112     if (CreateGraph(G, vertices, VR) != OK) {
113         return ERROR;
114     }
115
116     return OK;
117 }
```

Listing 29 邻接表读写

该函数首先打开文件，然后读取图的基本信息，包括图的类型、顶点数和边数。接着读取所有顶点的信息，并将其存储在顶点数组中。然后读取所有边的信息，并将其存储在边数组中。最后调用 CreateGraph 函数创建图的邻接表。注意这里再写入的时候对边进行了排序，保证了边的顺序是有序的，进而保证了邻接表在反复读写后形式是统一的。

17) 多邻接表管理

多邻接表管理函数与多链表管理函数类似，具体代码实现见附录。

2.4 系统测试

下面是参考测试用例进行的系统测试。

测试步骤 16：邻接表初始化

测试描述：初始化创建指定邻接表

```
5 线性表 2 3
8 集合 2
7 二叉树 1 3 0
6 无向图 2 0
```

预期结果

```
Choose your operations [0~25]: 18
Graph adjacency list:
Vertex 5 (线性表): 7 6
Vertex 8 (集合): 7
Vertex 7 (二叉树): 8 6 5
Vertex 6 (无向图): 7 5
```

实际结果

图 2-3 邻接表初始化测试结果对比

测试步骤 17：查找头节点

测试描述：查找关键字为 8 的头节点位置

```
你想操作第几个无向图:
1
你想查找的顶点的关键字是:
8
该顶点信息: 8 集合
```

预期结果

```
Choose your operations [0~25]: 3
Please input the key of the vertex to be located:
8
Vertex found at index: 1
Vertex information: Key: 8, Others: 集合
```

实际结果

图 2-4 头节点查找测试结果对比

测试步骤 18：第一邻接点

测试描述：查找 7 的下一邻接点

```
你想操作第几个无向图:
1
你想获得这个图中哪个顶点的第一邻接点:
7
第一邻接点信息: 8 集合
```

预期结果

```
Choose your operations [0~25]: 8
Please input the key of the vertex to be found:
7
First adjacent vertex: 1
Adjacent vertices: 8, 集合Press any key to continue...]
```

实际结果

图 2-5 邻接点查找测试结果对比

测试步骤 19：查找邻接点后继

测试描述：查找 7 的表节点 8 的下一邻接点

```
你想操作第几个无向图:
1
你想获得这个图中哪个顶点的哪个邻接点的下一邻接点:
7 8
下一邻接点信息: 6 无向图
```

预期结果

```
Please input the key of the vertex to be found:
7
8
Next adjacent vertex: 3
Adjacent vertices: 6, 无向图Press any key to continue...
```

实际结果

图 2-6 查找后继测试结果对比

测试步骤 20：头节点赋值

测试描述：将关键字为 6 的节点赋值为 9 无向图

```
Choose your operations to test:
Please input the key of the vertex to be modified:
6
Please input the new information of the vertex:
9 无向图
Vertex modified successfully!
```

预期结果 1

```
Graph adjacency list:
Vertex 5 (线性表): 7 9
Vertex 8 (集合): 7
Vertex 7 (二叉树): 8 9 5
Vertex 9 (无向图): 7 5
```

预期结果 2

图 2-7 头节点赋值结果

测试步骤 21：删除节点

测试描述：测试删除节点功能

```
树名      : 图1
5 线性表 7 二叉树 8 集合
```

预期结果 1

```
Graph adjacency list:
Vertex 5 (线性表): 7
Vertex 8 (集合): 7
Vertex 7 (二叉树): 8 5
```

预期结果 2

图 2-8 删除节点结果对比

测试步骤 22：深度优先搜索

测试描述：测试 DFS

```
请选择你的操作 [0~16]:11
树名      : 图1
5 线性表 7 二叉树 8 集合 6 无向图
```

预期结果

```
Choose your operations [0~25]: 11
Please input the key of the vertex to be traversed:
Key: 5, Others: 线性表
Key: 7, Others: 二叉树
Key: 8, Others: 集合
Key: 6, Others: 无向图
```

实际结果

图 2-9 DFS 测试结果对比

测试步骤 23：广度优先搜索

测试描述：测试 BFS

```
请选择你的操作 [0~16]:12
树名      : 图1
5 线性表 7 二叉树 8 集合 6 无向图
```

预期结果 1

```
Choose your operations [0~25]: 12
Please input the key of the vertex to be traversed:
1
Key: 5, Others: 线性表
Key: 7, Others: 二叉树
Key: 8, Others: 集合
Key: 6, Others: 无向图
BFS traversal completed successfully!
```

预期结果 2

图 2-10 BFS 测试结果对比

测试步骤 24：邻接表读写

测试描述：测试邻接表读写

```
Choose your operations [0~21]:13
Please input the file name to save the list: list_test.out
SaveList successfully!
Press any key to continue...]
```

实际结果 1

```
Choose your operations [0~21]:14
Please input the file name to load the list: list_test.out
Lists already exist, rewrite it? (y/n): y
LoadList successfully!
Press any key to continue...
```

实际结果 2

图 2-11 邻接表读写测试结果对比

测试步骤 25：多邻接表添加操作

测试描述：测试多邻接添加表操作功能是否正常工作

```
树名      : 图1
5 线性表 7 二叉树 8 集合 6 无向图
-----
树名      : 新图
5 线性表 7 二叉树 8 集合 6 无向图
```

预期结果

```
Graph group:
Graph 0: Graph1
Graph 1: Graph2
```

实际结果

图 2-12 多邻接表操作测试结果对比

测试描述：测试多邻接表删除功能是否正常工作

```
Choose your operations [0~25]: 20
Please input the name of the graph to be removed:
Graph1
Graph removed successfully!
```

实际结果 1

```
Graph group:
Graph 0: Graph2
```

实际结果 2

图 2-13 多链表操作测试结果对比

测试步骤 26：距离小于 k 的顶点

测试描述：测试距离小于 k 的顶点获取功能

输入：图名，集合，2↵

输出：二叉树↵

```
Please input the key of the vertex and the distance:
8 2
Vertices within distance 2 from vertex 8:
8 集合
7 二叉树
```

预期结果

实际结果

图 2-14 多链表操作测试结果对比 (认为节点到自身的距离为 0)

测试步骤 27：最短路径

测试描述：测试获取两节点最短路径长度功能

输入：图名，集合，线性表↵

输出：2↵

```
Choose your operations [0-25]: 10
Please input the start and end vertices:
5 8
Shortest path length: 2
Press any key to continue...|
```

预期结果

实际结果

图 2-15 最短路径测试结果对比

测试步骤 28：获取图的连通分量

测试描述：测试图的连通分量功能，初始状态：

输出：1↵

```
Choose your operations [0-25]: 17
Number of connected components: 1
Press any key to continue...|
```

预期结果 1

实际结果 1

图 2-16 链表排序测试结果对比

测试描述：断掉节点“集合”与“二叉树”之间的连边之后：

将集合与二叉树之间的边断开后↵

输入：图名↵

输出：2↵

```
Choose your operations [0-25]: 17
Number of connected components: 2
Press any key to continue...|
```

预期结果 2

实际结果 2

图 2-17 链表排序测试结果对比

2.5 实验小结

通过本次基于邻接表的图实现实验，我深入理解了图这一非线性数据结构的特性和应用价值。邻接表作为图的一种重要存储结构，特别适合表示稀疏图，能够有效节省存储空间，优化图的各种操作。

在实验过程中，我设计并实现了包括邻接表的基本操作和高级操作在内的一系列功能。基本操作包括邻接表的初始化、销毁、节点插入、节点删除、弧的插入和删除等；高级操作包括深度优先搜索、广度优先搜索、查找距离小于 k 的顶点、最短路径计算以及连通分量获取等。这些操作的实现帮助我深入理解了图论算法的核心思想和实际应用，同时加深了我对一些常用数据结构（如队列、栈）的理解和应用。

多邻接表管理系统的设计是本实验的一个创新点。我通过维护一个邻接表数组和相应的索引，实现了多个图的并发管理和操作切换。采用状态机和全局指针管理的方式，使得用户可以方便地在不同的图之间切换，同时保留各图的操作状态。这种设计模式不仅提高了代码的复用性，也增强了系统的可扩展性。

在实现过程中，我遇到了一些技术难点。例如，在删除顶点时，需要同时处理该顶点的所有入边和出边，确保图的一致性；在实现最短路径算法时，需要巧妙地应用 Dijkstra 算法处理图中的路径问题，通过解决这些问题，我加深了对图结构和算法的理解，提高了编程能力。

总的来说，这次实验不仅巩固了我对图数据结构的理论知识，还提升了我的算法设计和实现能力。通过自己动手实现各种图操作，我更加深入地理解了图的本质和其在解决实际问题中的重要作用。

3 课程的收获和建议

3.1 基于链式存储结构的线性表实现

通过学习链表的相关知识，我对链表相关操作有了更深入的理解。同时对多链表操作系统的构建，使我建立了对一种类似并发操作方法的基本认识。课程较全面的训练了我对链表操作的基本能力，内容较为完善。建议在此基础之上，增添适量的应用内容，提高对链表的灵活应用能力，而不仅仅培养是对抽象的数据结构的理解。

3.2 基于邻接表的图实现

通过学习邻接表的相关知识，我对图的存储结构和相关操作有了更深入的理解。通过实现一些基本的算法，如 BFS,DFS 等，我对这些算法的原理和应用有了更深入的认识。同样，建议在此基础之上，丰富图的应用内容，增加一些在综合问题利用图解决问题的案例。此外，建议实验课程开发一个自动检验程序，降低老师和学生的核查成本，提高课程效率。

参考文献

- [1] 袁凌, 祝建华, 许贵平, 李剑军, 周全. 数据结构——从概念到算法 [M]. [S.l.]: 人民邮电出版社, .

附录 A 基于顺序存储结构线性表实现的源程序

本项目由一下源文件构筑：

- Def.h - 定义了线性表的相关数据结构和常量。
- Lists.h - 定义了线性表的相关操作函数。
- Add-on-lists.h - 实现了线性表的附加操作函数。
- Lists-File-IO.h - 实现了线性表的文件读写操作函数。
- Lists-Multi.h - 实现了多链表的相关操作函数。
- main.cpp - 主驱动程序

以下是各文件的详细代码：

Def.h

```
1  #include "stdio.h"
2  #include "stdlib.h"
3  #define TRUE 1
4  #define FALSE 0
5  #define OK 1
6  #define ERROR 0
7  #define INFEASIBLE -1
8  #define OVERFLOW -2
9
10 typedef int status;
11 typedef int ElemType; //数据元素类型定义
12
13 #define LIST_INIT_SIZE 100
14 #define LISTINCREMENT 10
15
16 typedef struct{ //顺序表（顺序结构）的定义
17     ElemType * elem;
18     int length;
19     int listsize;
20 }SqlList;
21
22 typedef struct{ //线性表的管理表定义
23     struct { char name[30];
24             SqlList L;
25     } elem[10];
26     int length;
27     int listsize;
28 }LISTS;
```

Listing 30 Def.h

Lists.h

```
1
2 //-----Lists-Initialization-----//
3 status InitList(SqList* L)
4 // 线性表L不存在，构造一个空的线性表，返回OK，否则返回INFEASIBLE
5 。
6 {
7     if(L->elem == NULL)
8     {
9         // printf("Empty\n");
10        L->elem = (ElemType *)malloc(LIST_INIT_SIZE * sizeof(
11        ElemType));
12        L->length = 0;
13        L->listsize = LIST_INIT_SIZE;
14        return 1;
15    }
16
17    return -1;
18
19 }
20
21 status DestroyList(SqList* L)
22 // 如果线性表L存在，销毁线性表L，释放数据元素的空间，返回OK，否则
23 返回INFEASIBLE。
24 {
25     if(L->elem != NULL)
26     {
27         int *noob = (int *)malloc(114514*sizeof(int));
28         free(noob);
29         L->elem = 0;
30         return 1;
31     }
32
33     return -1;
34 }
35
36 status ClearList(SqList* L)
37 // 如果线性表L存在，删除线性表L中的所有元素，返回OK，否则返回
38 INFEASIBLE。
39 {
40     if(L->elem != NULL)
41     {
42         L->length = 0;
43         return 1;
44     }
45
46     return -1;
47 }
48
49 //-----Lists-Initialization-----//
```

```
48
49
50 //-----List-check-----//
51
52 status ListEmpty(Sqlist* L)
53 // 如果线性表L存在，判断线性表L是否为空，空就返回TRUE，否则返回
   FALSE；如果线性表L不存在，返回INFEASIBLE。
54 {
55     if(L->elem != NULL)
56         if(!L->length)
57             return TRUE;
58         else
59             return FALSE;
60
61     return -1;
62 }
63
64 status ListLength(Sqlist* L)
65 // 如果线性表L存在，返回线性表L的长度，否则返回INFEASIBLE。
66 {
67     if(L->elem)
68         return L->length;
69
70     return -1;
71 }
72
73
74 //-----List-Empty-check-----//
75
76 //-----List-Get-Operation-----//
77 status GetElem(Sqlist* L, int i, ElemType &e)
78 // 如果线性表L存在，获取线性表L的第i个元素，保存在e中，返回OK；如
   果i不合法，返回ERROR；如果线性表L不存在，返回INFEASIBLE。
79 {
80     if(L->elem)
81         if(i <= L->length && i >= 1)
82         {
83             e = *(L->elem + i - 1);
84             return TRUE;
85         }
86         else
87             return ERROR;
88
89     return -1;
90 }
91
92
93 int LocateElem(Sqlist* L, ElemType e)
94 // 如果线性表L存在，查找元素e在线性表L中的位置序号并返回该序号；
   如果e不存在，返回0；当线性表L不存在时，返回INFEASIBLE（即-1
   ） 。
95 {
96     if(L->elem)
97     {
98         int i = 0;
```

```
99     while(i < L->length)
100     {
101         if(*(L->elem + i) == e)
102             return (i + 1);
103         i++;
104     }
105     return FALSE;
106 }
107
108 return -1;
109
110 }
111
112 status PriorElem(SqList* L, ElemType e, ElemType &pre)
113 // 如果线性表L存在, 获取线性表L中元素e的前驱, 保存在pre中, 返回OK
114 // ; 如果没有前驱, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。
115 {
116     if(L->elem)
117     {
118         int i = 1;
119         while(i < L->length)
120         {
121             if(*(L->elem + i) == e)
122             {
123                 pre = (*(L->elem + i - 1));
124                 return TRUE;
125             }
126             i++;
127         }
128         return FALSE;
129     }
130
131     return -1;
132 }
133
134 status NextElem(SqList* L, ElemType e, ElemType &next)
135 // 如果线性表L存在, 获取线性表L元素e的后继, 保存在next中, 返回OK
136 // ; 如果没有后继, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。
137 {
138     if(L->elem)
139     {
140         int i = 0;
141         while(i < L->length - 1)
142         {
143             if(*(L->elem + i) == e)
144             {
145                 next = (*(L->elem + i + 1));
146                 return TRUE;
147             }
148             i++;
149         }
150         return FALSE;
151     }
```

```

152     }
153
154     return -1;
155
156 }
157
158 status ListDelete(SqList *L,int i,ElemType &e)
159 // 如果线性表L存在，删除线性表L的第i个元素，并保存在e中，返回OK；
    当删除位置不正确时，返回ERROR；如果线性表L不存在，返回
    INFEASIBLE。
160 {
161     if(L->elem)
162     {
163         if(i > 0 && i <= L->length)
164         {
165             int count = i - 1;
166             e = *(L->elem + count);
167             while(count < L->length - 1)
168             {
169                 *(L->elem + count) = *(L->elem + count + 1);
170                 count++;
171             }
172             L->length--;
173             return TRUE;
174         }
175
176         return FALSE;
177     }
178
179     return -1;
180
181 }
182
183 //-----List-Elem-Operation-----//
184
185 //-----Lists-Insert-----//
186
187 status ListInsert(SqList* L,int i,ElemType e)
188 // 如果线性表L存在，将元素e插入到线性表L的第i个元素之前，返回OK；
    当插入位置不正确时，返回ERROR；如果线性表L不存在，返回
    INFEASIBLE。
189 {
190     if(L->elem)
191     {
192         if(L->length == L->listsize)
193         {
194             //printf("Current size is insufficient,the current
195             length is %d, listsize is %d\n",L->length,L->listsize);
196             L->elem = (ElemType *)realloc(L->elem, (L->listsize +
197             LIST_INIT_SIZE) * sizeof(ElemType));
198             L->listsize += LIST_INIT_SIZE;
199         }
200         if(i > 0 && i <= L->length)
    
```

```
201         int count = L->length + 1;
202         while(count > i)
203         {
204             *(L->elem + count - 1) = *(L->elem + count - 2);
205             count--;
206         }
207         *(L->elem + count - 1) = e;
208         L->length++;
209         return TRUE;
210     }
211
212     if(i == L->length + 1)
213     {
214         *(L->elem + i - 1) = e;
215         L->length++;
216         return TRUE;
217     }
218
219     return FALSE;
220
221 }
222
223 return -1;
224
225 }
226
227 void InputList(Sqlist *L)
228 {
229     char c;
230     while((c=getchar())!='\0')
231     {
232         int count = 0;
233         ElemType e = (ElemType)(c - '\0');
234         ListInsert(L, count, e);
235     }
236 }
237 //-----Lists-Insert-----//
238
239 //-----Lists-I/O-----//
240 status ListTraverse(Sqlist* L)
241 // 如果线性表L存在，依次显示线性表中的元素，每个元素间空一格，返回OK；如果线性表L不存在，返回INFEASIBLE。
242 {
243     int idx = 0;
244     if(L->elem)
245     {
246         for(idx = 0; idx < L->length; idx++)
247         {
248             if(idx != L->length - 1)
249                 printf("%d ", L->elem[idx]);
250             else
251                 printf("%d", L->elem[idx]);
252         }
253
254         return OK;
```



```

255     }
256
257     return INFEASIBLE;
258
259 }
260
261 status SaveList(SqlList* L, char FileName[])
262 // 如果线性表L存在, 将线性表L的元素写到FileName文件中, 返回OK,
263 // 否则返回INFEASIBLE。
264 {
265     if(L->elem != NULL)
266     {
267         FILE *fin = fopen(FileName, "w");
268         int count = 0;
269         for(count = 0; count < L->length; count++)
270         {
271             fprintf(fin, "%d ", L->elem[count]);
272         }
273         fclose(fin);
274         return OK;
275     }
276
277     return INFEASIBLE;
278
279 }
280
281
282 status LoadList(SqlList *L, char FileName[])
283 // 如果线性表L不存在, 将FileName文件中的数据读入到线性表L中, 返回
284 // OK, 否则返回INFEASIBLE。
285 {
286     if(L->elem == NULL)
287     {
288         L->elem = (ElemType *)malloc(L->listsize * sizeof(
289 ElemType));
290         FILE *fout = fopen(FileName, "r");
291         int count = 0;
292         while (count < L->listsize && fscanf(fout, "%d", &L->elem
293 [count]) == 1) {
294             count++;
295         }
296         L->length = count;
297         fclose(fout);
298         return OK;
299     }
300
301     return INFEASIBLE;
302 }
303
304 //-----Lists-I/O-----//

```

Listing 31 Lists.h

Add-on-lists.h

```
1  #define UPPER 1000000
2  #define LOWER -1000000
3
4  //计算线性表L的最大子序列和
5  ElemType MaxSubArray(SqList* L)
6  {
7      if(L->elem == NULL) return ERROR;
8      if(L->length == 0) return 0;
9
10     int length = L->length;
11     ElemType *sub_sum = (ElemType *)malloc(sizeof(ElemType) * (
length + 1));
12     ElemType max_sum = LOWER, min_sum = UPPER;
13
14     //生成前缀和
15     sub_sum[0] = L->elem[0];
16     for(int i = 1; i < length; i++)
17     {
18         sub_sum[i] = sub_sum[i - 1] + L->elem[i];
19     }
20
21     for(int i = 0; i < length; i++)
22     {
23         for(int j = 0; j < i; j++)
24         {
25             max_sum = (max_sum < sub_sum[i] - sub_sum[j]) ?
sub_sum[i] - sub_sum[j]:max_sum;
26         }
27     }
28
29     free(sub_sum);
30     return max_sum;
31 }
32
33 int SubArrayNum(SqList* L, int k)
34 {
35     if(L->elem == NULL) return ERROR;
36     if(L->length == 0) return 0;
37
38     int length = L->length;
39     int *sub_sum = (int *)malloc(sizeof(int) * (length + 1));
40     int count = 0;
41
42     //生成前缀和
43     sub_sum[0] = L->elem[0];
44     for(int i = 1; i < length; i++)
45         sub_sum[i] = sub_sum[i - 1] + L->elem[i];
46
47     for(int i = 0; i < length; i++)
48         for(int j = i + 1; j < length; j++)
49             if(sub_sum[j] - sub_sum[i] == k)
```

```
50         count++;
51
52     free(sub_sum);
53     return count;
54 }
55
56 //顺序表排序
57 void SortList(Sqlist* L)
58 {
59     if(L->elem == NULL)
60     {
61         fprintf(stderr, "Error: List is NULL\n");
62         return;
63     }
64
65     for(int i = 0; i < L->length - 1; i++)
66     {
67         for(int j = i + 1; j < L->length; j++)
68         {
69             if(L->elem[i] > L->elem[j])
70             {
71                 int temp = L->elem[i];
72                 L->elem[i] = L->elem[j];
73                 L->elem[j] = temp;
74             }
75         }
76     }
77 }
```

Listing 32 Add-on-lists.h

Lists-File-IO.h

```
1     void ListFWrite(Sqlist *L, char *filename)
2     {
3         FILE *fp = fopen(filename, "w");
4         if (fp == NULL) {
5             fprintf(stderr, "Error opening file!\n");
6             return;
7         }
8
9         fprintf(fp, "%d\n", L->length);
10        for (int i = 0; i < L->length; i++) {
11            fprintf(fp, "%d ", L->elem[i]);
12        }
13        fclose(fp);
14    }
15
16    void ListFRead(Sqlist *L, char *filename)
17    {
18        L->elem = NULL;
19    }
```

```
20 FILE *fp = fopen(filename, "r");
21 if (fp == NULL) {
22     fprintf(stderr, "Error opening file!\n");
23     return;
24 }
25
26 fscanf(fp, "%d", &L->length);
27 L->listsize = LIST_INIT_SIZE * (L->length / LIST_INIT_SIZE +
28 1);
29 L->elem = (ElemType *)malloc(sizeof(ElemType) * L->length);
30 for (int i = 0; i < L->length; i++) {
31     fscanf(fp, "%d", &L->elem[i]);
32 }
33 fclose(fp);
34 }
```

Listing 33 Lists-File-IO.h

Lists-Multi.h

```
1
2 status InitMultiLists(LISTS *Lists)
3 // 初始化Lists, 创建一个空的线性表Lists, 返回OK, 否则返回
4 // INFEASIBLE。
5 {
6     Lists->elem = (LinkList *)malloc(sizeof(LinkList) *
7     MAX_LISTS_NUM);
8     Lists->name = (char **)malloc(sizeof(char *) * MAX_LISTS_NUM)
9     ;
10     for(int i = 0; i < MAX_LISTS_NUM; i++)
11     {
12         Lists->elem[i] = NULL;
13         Lists->name[i] = (char *)malloc(sizeof(char) * 20); // 假
14         设每个列表名称不超过20个字符
15     }
16     if(Lists->elem == NULL)
17         return INFEASIBLE;
18
19     Lists->length = 0;
20     Lists->listsize = MAX_LISTS_NUM;
21
22     return OK;
23 }
24
25 status DestroyMultiLists(LISTS Lists)
26 {
27     // 销毁Lists, 释放数据元素的空间, 返回OK, 否则返回INFEASIBLE
28     。
29     if(Lists.elem == NULL)
30         return INFEASIBLE;
31 }
```

```
28     for(int i = 0; i < Lists.length; i++)
29     {
30         DestroyList(&Lists.elem[i]);
31     }
32
33     free(Lists.elem);
34     Lists.elem = NULL;
35
36     return OK;
37 }
38
39 status CheckDuplicate(LISTS Lists, char *ListName)
40 {
41     // 检查Lists中是否存在名称为ListName的线性表, 存在返回TRUE,
42     // 否则返回FALSE
43     if(Lists.elem == NULL)
44         return INFEASIBLE;
45
46     for(int i = 0; i < Lists.length; i++)
47     {
48         if(strcmp(Lists.name[i], ListName) == 0)
49             return TRUE;
50     }
51
52     return FALSE;
53 }
54
55 status AddList(LISTS *Lists, char *ListName)
56 {
57     if(Lists->elem == NULL)
58         return INFEASIBLE;
59
60     if(Lists->length == MAX_LISTS_NUM)
61         return OVERFLOW;
62
63     //check if the list name already exists
64     if(CheckDuplicate(*Lists, ListName) == TRUE)
65         return ERROR;
66
67     printf("Add the list %s, list id: %d\n", ListName, Lists->
68         length + 1);
69     strcpy(Lists->name[Listes->length], ListName);
70     InitList(&Lists->elem[Listes->length]);
71     Lists->length++;
72
73     return OK;
74 }
75
76 status RemoveList(LISTS *Lists, char *ListName)
77 {
78     // 删除Lists中一个名称为ListName的线性表
79     if(Lists->elem == NULL)
80         return INFEASIBLE;
```

```
81     int idx = 0;
82     while(idx < Lists->length)
83     {
84         if(strcmp(Lists->name[idx], ListName) == 0)
85             break;
86         idx++;
87     }
88
89     if(idx == Lists->length)
90         return ERROR;
91
92     DestroyList(&Lists->elem[idx]);
93
94     for(int i = idx; i < Lists->length - 1; i++)
95     {
96         strcpy(Lists->name[i], Lists->name[i + 1]);
97         Lists->elem[i] = Lists->elem[i + 1];
98     }
99
100    Lists->length--;
101    Lists->elem[Listes->length] = NULL; // 清空最后一个元素的指针
102    return OK;
103 }
104
105 int LocateList(LISTS Lists, char *ListName)
106 {
107     // 在Lists中查找一个名称为ListName的线性表，成功返回逻辑序
108     // 号，否则返回0
109     if(Lists.elem == NULL)
110         return INFEASIBLE;
111
112     int idx = 0;
113     while(idx < Lists.length)
114     {
115         if(strcmp(Lists.name[idx], ListName) == 0)
116             break;
117         idx++;
118     }
119
120     if(idx == Lists.length && strcmp(Lists.name[idx], ListName) != 0)
121         return ERROR;
122
123     return idx;
124 }
125
126 void PrintLists(LISTS Lists)
127 {
128     // 打印所有线性表的名称
129     if(Lists.elem == NULL)
130     {
131         printf("No lists exist!\n");
132         return;
133     }
```

```
134     for(int i = 0; i < Lists.length; i++)
135     {
136         printf("List %d: %s\n", i + 1, Lists.name[i]);
137     }
138
139     return;
140 }
```

Listing 34 Lists-Multi.h

main.cpp

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #include "Def.h"
6  #include "Lists.h"
7  #include "Lists-File-IO.h"
8  #include "Lists-Multi.h"
9  #include "Add-on-lists.h"
10
11 int main(void){
12
13     SqList *L;
14     SqList singleL;
15     singleL.elem = NULL;
16
17     int op=1;
18
19     int multilist = 1;
20     int lidx = 0;
21     LISTS Lists;
22     Lists.length = 0;
23     Lists.listsize = LIST_INIT_SIZE;
24
25     while(multilist)
26     {
27         printf("-----Mode Selecting-----\n");
28         printf("      1.Single-Lists\n      2.Multi-Lists\n      0.exit\n");
29         scanf("%d",&multilist);
30         if(multilist == 0) break;
31         op = 1;
32
33         while(op){
34
35             system("cls");
36             if(multilist == 2)
37             {
38                 printf("Multi-Lists Operating Mode:\n\n");
39                 if(lidx < 0)
```

华中科技大学课程实验报告

```
40         {
41             printf("No list now!");
42         }
43         else
44         {
45             printf("Current List Name: %s\n",Lists.elem[lidx].
name);
46             L = &Lists.elem[lidx].L;
47         }
48     }
49
50     else
51         L = &singleL;
52     printf("\n\n");
53     printf("          Menu for Linear Table On Sequence
Structure \n");
54     printf("-----\n");
55     printf("          1. InitList          7. LocateElem\n");
56     printf("          2. DestroyList       8. PriorElem\n");
57     printf("          3. ClearList         9. NextElem \n");
58     printf("          4. ListEmpty         10. ListInsert\n");
59     printf("          5. ListLength        11. ListDelete\n");
60     printf("          6. GetElem           12. ListTraverse\n");
61     ;
62     printf("-----\n");
63     printf("          Menu for Add-on lists function\n");
64     printf("-----\n");
65     printf("          13. ListFWrite       14. ListFRead\n");
66     printf("          15. MaxsubArray      16. SubArrayNumt\n");
67     printf("          17. Sortlist         \n");
68     printf("-----\n");
69     printf("          Menu for Multi-Lists Operating Mode\n");
70     printf("-----\n");
71     printf("          18. AddList          19. RemoveList\n");
72     printf("          20. LocateList       21. ShowList \n");
73     printf("          0. Exit\n");
74     printf("-----\n");
75     printf("          Choose operation:[0~20]:");
76     scanf("%d",&op);
77
78     if(!(*L).elem && 0)
79         printf("error!\n");
80
81     switch(op)
82     {
83     case 1:
84         if(InitList(L) == OK)
85             printf("InitList OK\n");
86         else
```



```
86         printf("InitList ERROR\n");
87
88         //printf("Pre input the list?\n");
89         char c;
90         //if((c = getchar()) == '1')
91         //InputList(L);
92         break;
93
94     case 2:
95         if(DestroyList(L) == OK)
96             printf("DestroyList OK\n");
97         else
98             printf("DestroyList ERROR\n");
99         break;
100
101     case 3:
102         if(ClearList(L) == OK)
103             printf("ClearList OK\n");
104         else
105             printf("ClearList ERROR\n");
106         break;
107
108     case 4:
109         if(ListEmpty(L) == TRUE)
110             printf("List is empty!\n");
111         if(ListEmpty(L) == FALSE)
112             printf("List is not empty!\n");
113         if(ListEmpty(L) == INFEASIBLE)
114             printf("List is not exist!\n");
115         break;
116
117     case 5:
118         if((ListLength(L)) != INFEASIBLE)
119             printf("List length is %d\n",ListLength(L));
120         else
121             printf("List is not exist!\n");
122         break;
123
124     case 6:
125     {
126         int i; ElemType e;
127         printf("Please input the location of the
128 element:");
129         scanf("%d",&i);
130         if(GetElem(L,i,e) == OK)
131             printf("GetElem OK, the element is %d\n",e);
132         else
133             printf("GetElem ERROR\n");
134     }
135     break;
136
137     case 7:
138     {
139         int i; ElemType e;
140         printf("Please input the element to be located:
```

```

140         ");
141         scanf("%d",&e);
142         if(LocateElem(L,e) != 0)
143             printf("LocateElem OK, the element is %d\n",
LocateElem(L,e));
144         else
145             printf("LocateElem ERROR\n");
146     }
147     break;
148
149     case 8:
150     {
151         int i; ElemType e;
152         printf("Please input the element to be located:
153         ");
154         scanf("%d",&i);
155         if(PriorElem(L,i,e) == OK)
156             printf("PriorElem OK, the element is %d\n",e
157         );
158         else
159             printf("PriorElem ERROR\n");
160     }
161     break;
162
163     case 9:
164     {
165         int i; ElemType e;
166         printf("Please input the element to be located:
167         ");
168         scanf("%d",&i);
169         if(NextElem(L,i,e) == OK)
170             printf("NextElem OK, the element is %d\n",e)
171         ;
172         else
173             printf("NextElem ERROR\n");
174     }
175     break;
176
177     case 10:
178     {
179         int i; ElemType e;
180         printf("Please input the location to be
181         inserted:");
182         scanf("%d",&i);
183         printf("Please input the element to be inserted
184         :");
185         scanf("%d",&e);
186         if(ListInsert(L,i,e) == OK)
187             printf("ListInsert OK\n");
188         else
189             printf("ListInsert ERROR\n");
190     }
191     break;
192
193     case 11:

```

```

187         {
188             int i; ElemType e;
189             printf("Please input the location to be deleted
190 :");
191             scanf("%d",&i);
192             if(ListDelete(L,i,e) == OK)
193                 printf("ListDelete OK, the element is %d\n",
194 e);
195             else
196                 printf("ListDelete ERROR\n");
197         }
198         break;
199     case 12:
200         if(ListTraverse(L) == OK)
201             printf("\nListTraverse OK\n");
202         else
203             printf("ListTraverse ERROR\n");
204         break;
205     case 13:
206         {
207             char filename[20];
208             printf("Please input the file name:");
209             scanf("%s",filename);
210             ListFWrite(L,filename);break;
211         }
212     case 14:
213         {
214             char filename[20];
215             printf("Please input the file name:");
216             scanf("%s",filename);
217             ListFRead(L,filename);break;
218         }
219     case 15:
220         {
221             ElemType e;
222             e = MaxSubArray(L);
223             printf("MaxsubArray OK, the element is %d\n",e)
224 ;
225             break;
226         }
227     case 16:
228         {
229             int k;
230             printf("Input k:");
231             scanf("%d",&k);
232             int count = SubArrayNum(L,k);
233             printf("SubArrayNum OK, the element is %d\n",
234 count);
235         }

```

```

238         break;
239     }
240
241     case 17:
242     {
243         SortList(L);
244         printf("SortList OK\n");
245         for(int i = 0; i < L->length; i++)
246         {
247             if(i != L->length - 1)
248                 printf("%d ",L->elem[i]);
249             else
250                 printf("%d",L->elem[i]);
251         }
252         printf("\n");
253         break;
254     }
255
256     case 18:
257     {
258         printf("Please input the name of the lists:\n")
;
259         char nameadd[30];
260         scanf("%s",nameadd);
261         AddList(&Lists, nameadd);
262         break;
263     }
264
265     case 19:
266     {
267         printf("Please input the name of the list to be
removed:\n");
268         char namerm[30];
269         scanf("%s", namerm);
270         RemoveList(&Lists, namerm);
271         break;
272     }
273
274     case 20:
275     {
276         printf("Please input the list name to be
located:\n");
277         char namelc[30];
278         scanf("%s",namelc);
279         int id = LocateList(&Lists, namelc);
280         printf("The located list id is:%d\n",id);
281         printf("Switch to the new list? (1/0)\n");
282         bool sw = false;
283         scanf("%d",&sw);
284         if(sw)
285             lidx = id;
286         break;
287     }
288
289     case 21:

```

```
290         {
291             ShowList(&Lists);
292         }
293
294     }
295
296     if(op)
297     {
298         getchar();
299         printf("Continue?(\\n)");
300         getchar();
301     }
302
303 }
304
305 }
306 }
```

Listing 35 main.cpp

附录 B 基于链式存储结构线性表实现的源程序

本项目由以下源文件构筑：

- def.h - 定义了线性表的相关数据结构和常量。
- Link-Lists.h - 定义了线性表的相关操作函数。
- Link-Lists-Addons.h - 实现了线性表的附加操作函数。
- Link-Lists-IO.h - 实现了线性表的文件读写操作函数。
- Lins-Lists-Multi.h - 实现了多链表的相关操作函数。
- main.cpp - 主驱动程序

以下是各文件的详细代码：

def.h

```
1  #include "stdio.h"
2  #include "stdlib.h"
3  #include <string.h>
4  #define TRUE 1
5  #define FALSE 0
6  #define OK 1
7  #define ERROR -32766
8  #define INFEASIBLE -3
9  #define OVERFLOW -2
10 #define MAX_LISTS_NUM 20
11
12 typedef int status;
13 typedef int ElemType; //数据元素类型定义
14
15 #define LIST_INIT_SIZE 100
16 #define LISTINCREMENT 10
17 typedef int ElemType;
18 typedef struct LNode{ //单链表（链式结构）结点的定义
19     ElemType data;
20     struct LNode *next;
21 }LNode,*LinkList;
22
23 typedef struct LIST {
24     LinkList *elem = NULL;
25     char **name;
26     int length;
27     int listsize;
28 } LIST;
```

Listing 36 def.h

Link-Lists.h

```
1  // #include "def.h"
2
3  status InitList(LinkList *L)
4  // 线性表L不存在, 构造一个空的线性表, 返回OK, 否则返回INFEASIBLE
5  。
6  {
7      if(*L == NULL)
8      {
9          *L = (LNode *)malloc(sizeof(LNode));
10         (*L) -> next = NULL;
11         return OK;
12     }
13     return INFEASIBLE;
14 }
15
16 status DestroyList(LinkList *L)
17 // 如果线性表L存在, 销毁线性表L, 释放数据元素的空间, 返回OK, 否则
18 返回INFEASIBLE。
19 {
20     if(*L != NULL)
21     {
22         LNode *start = *L;
23         LNode *temp = *L;
24         while((*L) -> next != NULL)
25         {
26             *L = (*L) -> next;
27             free(temp);
28             temp = *L;
29         }
30         *L = NULL;
31         free(temp);
32         return OK;
33     }
34     return INFEASIBLE;
35 }
36
37 status ClearList(LinkList *L)
38 // 如果线性表L存在, 删除线性表L中的所有元素, 返回OK, 否则返回
39  INFEASIBLE。
40 {
41     if(*L == NULL) {
42         return INFEASIBLE; // 链表不存在
43     }
44     LNode *p = (*L) -> next; // p指向第一个数据节点
45     LNode *temp;
46
47     while(p != NULL) {
48         temp = p; // 保存当前节点
```

```
49     p = p->next;        // p移动到下一个节点
50     free(temp);         // 释放当前节点
51 }
52
53 (*L)->next = NULL;     // 头节点的next指针置为NULL
54 return OK;
55 }
56
57 status ListEmpty(LinkList *L)
58 // 如果线性表L存在, 判断线性表L是否为空, 空就返回TRUE, 否则返回
59 // FALSE; 如果线性表L不存在, 返回INFEASIBLE。
60 {
61     if(*L == NULL)
62         return INFEASIBLE;
63
64     if((*L) -> next == NULL)
65         return TRUE;
66
67     return FALSE;
68 }
69
70 int ListLength(LinkList *L)
71 // 如果线性表L存在, 返回线性表L的长度, 否则返回INFEASIBLE。
72 {
73     if(*L == NULL)
74         return INFEASIBLE;
75
76     int length = 0;
77     LNode *temp = *L;
78     while(temp -> next != NULL)
79     {
80         temp = temp -> next;
81         length++;
82     }
83
84     return length;
85 }
86
87 status GetElem(LinkList *L, int i, ElemType *e)
88 // 如果线性表L存在, 获取线性表L的第i个元素, 保存在e中, 返回OK; 如
89 // 果i不合法, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。
90 {
91     if(*L == NULL)
92         return INFEASIBLE;
93
94     LNode * temp = *L;
95     int count = 0;
96     while(temp -> next != NULL && count < i)
97     {
98         temp = temp -> next;
99         count++;
100     }
101
102     if(count < i - 1 || count == 0) return ERROR;
```



```
102     *e = temp -> data;
103
104     return OK;
105 }
106
107 status LocateElem(LinkList *L, ElemType e)
108 // 如果线性表L存在, 查找元素e在线性表L中的位置序号; 如果e不存在,
109 // 返回ERROR; 当线性表L不存在时, 返回INFEASIBLE。
110 {
111     if(*L == NULL)
112         return INFEASIBLE;
113
114     int idx = 0;
115     LNode *temp = *L;
116     while(temp != NULL)
117     {
118         if(temp -> data == e)
119             return idx;
120         temp = temp -> next;
121         idx++;
122     }
123
124     if(temp -> data != e)
125         return ERROR;
126
127     return ERROR; // 补充返回值
128 }
129
130 status PriorElem(LinkList *L, ElemType e, ElemType *pre)
131 // 如果线性表L存在, 获取线性表L中元素e的前驱, 保存在pre中, 返回OK
132 // 如果没有前驱, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE。
133 {
134     if(*L == NULL)
135         return INFEASIBLE;
136
137     // 如果链表为空(仅有头节点)
138     if((*L)->next == NULL)
139         return ERROR;
140
141     LNode *current = (*L)->next;
142     LNode *prior = *L;
143
144     while(current != NULL)
145     {
146         if(current->data == e && prior != *L)
147         {
148             *pre = prior->data;
149             return OK;
150         }
151         prior = current;
152         current = current->next;
153     }
154
155     // 如果没找到元素e或元素e是第一个元素(无前驱)
156     return ERROR;
```

```
155 }
156
157 status NextElem(LinkList *L, ElemType e, ElemType *next)
158 // 如果线性表L存在, 获取线性表L中元素e的后继, 保存在next中, 返回
    OK; 如果没有后继, 返回ERROR; 如果线性表L不存在, 返回INFEASIBLE
    。
159 {
160     if(*L == NULL)
161         return INFEASIBLE;
162
163     // 如果链表为空(仅有头节点)
164     if((*L)->next == NULL)
165         return ERROR;
166
167     LNode *current = (*L)->next;
168
169     while(current != NULL)
170     {
171         // 找到元素e
172         if(current->data == e)
173         {
174             // 检查是否有后继
175             if(current->next != NULL)
176             {
177                 *next = current->next->data;
178                 return OK;
179             }
180             else
181             {
182                 // 元素e是最后一个元素, 没有后继
183                 return ERROR;
184             }
185         }
186         current = current->next;
187     }
188
189     // 没有找到元素e
190     return ERROR;
191 }
192
193 status ListInsert(LinkList *L, int i, ElemType e)
194 // 如果线性表L存在, 将元素e插入到线性表L的第i个元素之前, 返回OK;
    当插入位置不正确时, 返回ERROR; 如果线性表L不存在, 返回
    INFEASIBLE。
195 {
196     if(*L == NULL)
197         return INFEASIBLE;
198
199     int count = 0;
200     LNode *prior = *L;
201     LNode *current = (*L) -> next;
202     while(current != NULL && count < i - 1)
203     {
204         prior = current;
205         current = current -> next;
```

```

206         count++;
207     }
208
209     if(count != i - 1) return ERROR;
210
211     LNode *insert = (LNode *)malloc(sizeof(LNode));
212     insert->data = e;
213     prior->next = insert;
214     insert->next = current;
215     return OK;
216 }
217
218 status ListDelete(LinkList *L, int i, ElemType *e)
219 // 如果线性表L存在, 删除线性表L的第i个元素, 并保存在e中, 返回OK;
    // 当删除位置不正确时, 返回ERROR; 如果线性表L不存在, 返回
    INFEASIBLE。
220 {
221     // 检查链表是否存在
222     if(*L == NULL)
223         return INFEASIBLE;
224
225     // 检查删除位置是否合法
226     if(i <= 0)
227         return ERROR;
228
229     int count = 0;
230     LNode *prior = *L; // 从头节点开始
231
232     // 定位到第i-1个位置
233     while(prior != NULL && count < i-1)
234     {
235         prior = prior->next;
236         count++;
237     }
238
239     // 检查定位是否成功以及要删除的位置是否有效
240     if(prior == NULL || prior->next == NULL)
241         return ERROR; // i大于链表长度或链表为空
242
243     // 删除节点
244     LNode *current = prior->next; // 要删除的节点
245     prior->next = current->next;
246
247     // 保存被删除节点的数据
248     *e = current->data;
249
250     // 释放节点内存
251     free(current);
252     return OK;
253 }

```

Listing 37 Link-Lists.h

Link-Lists-Addons.h

```
1 // #include "def.h"
2
3 status InitSequenceLists(LinkList *L)
4 {
5     LNode *temp = *L;
6
7     if(*L == NULL)
8     {
9         *L = (LinkList)malloc(sizeof(LNode));
10        if(*L == NULL)
11            return OVERFLOW;
12        (*L)->next = NULL;
13    }
14
15    ElemType e = 0;
16    while(scanf("%d", &e) == 1 && e != 0)
17    {
18        LNode *newNode = (LNode *)malloc(sizeof(LNode));
19        if(newNode == NULL)
20            return OVERFLOW;
21        newNode->data = e;
22        newNode->next = NULL;
23        (*L)->next = newNode;
24        (*L) = newNode; // 更新尾指针
25    }
26
27    (*L)->next = NULL; // 结束链表
28    *L = temp; // 恢复头指针
29    return OK;
30 }
31
32 status printLists(LinkList *L)
33 {
34     if(*L == NULL)
35         return INFEASIBLE;
36
37     LNode *temp = (*L)->next;
38     while(temp != NULL)
39     {
40         printf("%d ", temp->data);
41         temp = temp->next;
42     }
43     printf("\n");
44
45     return OK;
46 }
47
48
49 status reverseList(LinkList *L)
50 {
51     if(*L == NULL)
```

```
52     return INFEASIBLE;
53
54     LNode *current = (*L)->next->next; // 从第一个数据节点开始
55     LNode *prev = (*L)->next;
56     LNode *next = NULL;
57
58     while(current != NULL)
59     {
60         next = current->next;
61         current->next = prev;
62         prev = current;
63         current = next;
64     }
65
66     (*L)->next->next = NULL;
67     (*L)->next = prev;
68
69     return OK;
70 }
71
72 int DeleteNodefromEnd(LinkList *L, int n)
73 {
74     if(*L == NULL || (*L)->next == NULL)
75         return INFEASIBLE;
76
77     if(n <= 0)
78         return ERROR; // 添加参数检查
79
80     LNode *dummy = *L; // 使用头节点作为虚拟节点
81     LNode *fast = dummy;
82     LNode *slow = dummy;
83
84     // fast先走n步 (注意是n而非n-1)
85     for(int i = 0; i < n; i++)
86     {
87         if(fast->next == NULL)
88             return ERROR; // n大于链表长度
89         fast = fast->next;
90     }
91
92     // fast和slow一起走, 直到fast到达链表末尾
93     while(fast->next != NULL)
94     {
95         slow = slow->next;
96         fast = fast->next;
97     }
98
99     // 此时slow指向要删除节点的前一个节点
100    ElemType e = slow->next->data; // 要删除节点的值
101    LNode *temp = slow->next;
102    slow->next = slow->next->next;
103    free(temp);
104
105    return e;
106 }
```

```
107
108 status SortList(LinkList *L)
109 {
110     if(*L == NULL || (*L)->next == NULL)
111         return INFEASIBLE;
112
113     LNode *current = (*L)->next;
114     LNode *next = NULL;
115     LNode *sorted = NULL;
116
117     while(current != NULL)
118     {
119         next = current->next; // 保存下一个节点
120         if(sorted == NULL || sorted->data >= current->data) // 插
入到头部
121         {
122             current->next = sorted;
123             sorted = current;
124         }
125         else // 插入到中间或尾部
126         {
127             LNode *temp = sorted;
128             while(temp->next != NULL && temp->next->data <
current->data)
129             {
130                 temp = temp->next;
131             }
132             current->next = temp->next;
133             temp->next = current;
134         }
135         current = next; // 继续处理下一个节点
136     }
137
138     (*L)->next = sorted; // 更新头指针
139
140     return OK;
141 }
```

Listing 38 Link-Lists-Addons.h

Link-Lists-IO.h

```
1 // #include "def.h"
2
3 status SaveList(LinkList *L, char FileName[])
4 // 如果线性表L存在，将线性表L的元素写到FileName文件中，返回OK，
   否则返回INFEASIBLE。
5 {
6     if(*L == NULL)
7         return INFEASIBLE;
8
9     FILE *fout = fopen(FileName, "w");
```

```
10     if(fout == NULL)
11         return ERROR;
12
13     LNode *temp = (*L)->next;
14     while(temp != NULL)
15     {
16         fprintf(fout, "%d ", temp->data);
17         temp = temp->next;
18     }
19
20     fclose(fout);
21     return OK;
22 }
23
24 status LoadList(LinkList *L, char FileName[])
25 // 如果线性表L不存在, 将FileName文件中的数据读入到线性表L中, 返回
26 // OK, 否则返回INFEASIBLE。
27 {
28     if(*L != NULL) // 如果链表已存在, 返回INFEASIBLE
29     {
30         printf("Lists already exist, rewrite it? (y/n): ");
31         char choice;
32         scanf(" %c", &choice);
33         if(choice != 'y' && choice != 'Y')
34             return INFEASIBLE; // 用户选择不重写
35     }
36
37     FILE *fin = fopen(FileName, "r");
38     if(fin == NULL)
39         return ERROR;
40
41     DestroyList(L); // 销毁原有链表
42
43     // 创建头节点
44     *L = (LinkList)malloc(sizeof(LNode));
45     if(*L == NULL) {
46         fclose(fin);
47         return ERROR;
48     }
49     (*L)->next = NULL;
50
51     LNode *tail = *L; // 尾指针, 用于尾插法
52     int value;
53
54     // 读取文件中的数据并创建链表节点
55     while(fscanf(fin, "%d", &value) == 1)
56     {
57         LNode *newNode = (LNode*)malloc(sizeof(LNode));
58         if(newNode == NULL) {
59             fclose(fin);
60             // 释放已分配的链表
61             LNode *current = *L;
62             while(current != NULL) {
63                 LNode *temp = current;
64                 current = current->next;
```

```
64         free(temp);
65     }
66     *L = NULL;
67     return ERROR;
68 }
69
70     newNode->data = value;
71     newNode->next = NULL;
72     tail->next = newNode;
73     tail = newNode;
74 }
75
76 fclose(fin);
77 return OK;
78 }
```

Listing 39 Link-Lists-IO.h

Link-Lists-Multi.h

```
1
2 status InitMultiLists(LISTS *Lists)
3 // 初始化Lists, 创建一个空的线性表Lists, 返回OK, 否则返回
4 // INFEASIBLE。
5 {
6     Lists->elem = (LinkList *)malloc(sizeof(LinkList) *
7     MAX_LISTS_NUM);
8     Lists->name = (char **)malloc(sizeof(char *) * MAX_LISTS_NUM)
9     ;
10     for(int i = 0; i < MAX_LISTS_NUM; i++)
11     {
12         Lists->elem[i] = NULL;
13         Lists->name[i] = (char *)malloc(sizeof(char) * 20); // 假
14         设每个列表名称不超过20个字符
15     }
16     if(Lists->elem == NULL)
17         return INFEASIBLE;
18
19     Lists->length = 0;
20     Lists->listsize = MAX_LISTS_NUM;
21
22     return OK;
23 }
24
25 status DestroyMultiLists(LISTS Lists)
26 {
27     // 销毁Lists, 释放数据元素的空间, 返回OK, 否则返回INFEASIBLE
28     。
29     if(Lists.elem == NULL)
30         return INFEASIBLE;
```



```
28     for(int i = 0; i < Lists.length; i++)
29     {
30         DestroyList(&Lists.elem[i]);
31     }
32
33     free(Lists.elem);
34     Lists.elem = NULL;
35
36     return OK;
37 }
38
39 status CheckDuplicate(LISTS Lists, char *ListName)
40 {
41     // 检查Lists中是否存在名称为ListName的线性表, 存在返回TRUE,
42     // 否则返回FALSE
43     if(Lists.elem == NULL)
44         return INFEASIBLE;
45
46     for(int i = 0; i < Lists.length; i++)
47     {
48         if(strcmp(Lists.name[i], ListName) == 0)
49             return TRUE;
50     }
51
52     return FALSE;
53 }
54
55 status AddList(LISTS *Lists, char *ListName)
56 {
57     if(Lists->elem == NULL)
58         return INFEASIBLE;
59
60     if(Lists->length == MAX_LISTS_NUM)
61         return OVERFLOW;
62
63     //check if the list name already exists
64     if(CheckDuplicate(*Lists, ListName) == TRUE)
65         return ERROR;
66
67     printf("Add the list %s, list id: %d\n", ListName, Lists->
length + 1);
68     strcpy(Lists->name[Lists->length], ListName);
69     InitList(&Lists->elem[Lists->length]);
70     Lists->length++;
71
72     return OK;
73 }
74
75 status RemoveList(LISTS *Lists, char *ListName)
76 {
77     // 删除Lists中一个名称为ListName的线性表
78     if(Lists->elem == NULL)
79         return INFEASIBLE;
80 }
```

```
81     int idx = 0;
82     while(idx < Lists->length)
83     {
84         if(strcmp(Lists->name[idx], ListName) == 0)
85             break;
86         idx++;
87     }
88
89     if(idx == Lists->length)
90         return ERROR;
91
92     DestroyList(&Lists->elem[idx]);
93
94     for(int i = idx; i < Lists->length - 1; i++)
95     {
96         strcpy(Lists->name[i], Lists->name[i + 1]);
97         Lists->elem[i] = Lists->elem[i + 1];
98     }
99
100     Lists->length--;
101     Lists->elem[Lists->length] = NULL; // 清空最后一个元素的指针
102     return OK;
103 }
104
105 int LocateList(LISTS Lists, char *ListName)
106 {
107     // 在Lists中查找一个名称为ListName的线性表，成功返回逻辑序
108     // 号，否则返回0
109     if(Lists.elem == NULL)
110         return INFEASIBLE;
111
112     int idx = 0;
113     while(idx < Lists.length)
114     {
115         if(strcmp(Lists.name[idx], ListName) == 0)
116             break;
117         idx++;
118     }
119
120     if(idx == Lists.length && strcmp(Lists.name[idx], ListName) != 0)
121         return ERROR;
122
123     return idx;
124 }
125
126 void PrintLists(LISTS Lists)
127 {
128     // 打印所有线性表的名称
129     if(Lists.elem == NULL)
130     {
131         printf("No lists exist!\n");
132         return;
133     }
```

```
134     for(int i = 0; i < Lists.length; i++)
135     {
136         printf("List %d: %s\n", i + 1, Lists.name[i]);
137     }
138
139     return;
140 }
```

Listing 40 Link-Lists-Multi.h

main.cpp

```
1  #include "def.h"
2  #include "Link-Lists.h"
3  #include "Link-Lists-IO.h"
4  #include "Link-Lists-Addons.h"
5  #include "Link-Lists-Multi.h"
6
7  int main()
8  {
9      int mode = 1;
10     int op = 1;
11
12     LinkList *L_operated = NULL;
13
14     LinkList L_single = NULL;
15
16     LISTS Lists;
17     InitMultiLists(&Lists);
18     int list_idx = 0;
19     int loc_idx = 0;
20     int length = 0;
21     int n;
22
23     ElemType deletedElem;
24
25     char fileName[30];
26     char Lists_name[30];
27
28     while(mode)
29     {
30         system("cls");      printf("\n\n");
31         printf("          Menu for Mode Selection \n");
32         printf("-----\n");
33         printf("          1. Single Linked List \n");
34         printf("          2. Multi Linked List \n");
35         printf("          0. Exit \n");
36         printf("-----\n\n");
37         printf("          Choose the mode from 0 to 2: ");
38         scanf("%d",&mode);
```

华中科技大学课程实验报告

```
39     L_operated = NULL; // 重置操作的链表指针
40
41     while(op)
42     {
43         if(mode == 2)
44         {
45             system("cls");    printf("\n\n");
46             printf("-----\n");
47             printf("          Multi-Lists Operation Mode:\n");
48             ;
49             printf("-----\n");
50             if(Lists.elem[list_idx] == NULL )
51             {
52                 L_operated = NULL;
53                 printf("          No lists\n");
54                 printf("-----\n");
55             }
56             else
57             {
58                 L_operated = &Lists.elem[list_idx];
59                 printf("          Current list: %s",
60 Lists.name[list_idx]);
61                 printf("      List id: %d\n", list_idx + 1);
62                 printf("-----\n");
63             }
64         }
65         else if(mode == 1)
66         {
67             L_operated = &L_single;
68             system("cls");    printf("\n\n");
69             printf("-----\n");
70             printf("          Single-Lists Operation Mode:\n");
71             );
72             printf("-----\n");
73         }
74         else if(mode == 0)
75         {
76             printf("Exiting...\n");
77             break;
78         }
79         else
80         {
81             printf("Invalid mode!\n");
82             continue;
83         }
84     }
85
86     // 在 while(op) 循环里面添加的代码
87     printf("          Menu for Linear Table On Link Structure
```

```

84         \n");
85         printf("-----\n");
86         // 单链表和多链表通用操作
87         printf("          1. InitList          7. LocateElem\n"
88 );
89         printf("          2. DestroyList       8. PriorElem\n")
90 ;
91         printf("          3. ClearList          9. NextElem \n")
92 ;
93         printf("          4. ListEmpty         10. ListInsert\n"
94 );
95         printf("          5. ListLength        11. ListDelete\n"
96 );
97         printf("          6. GetElem           12. ListTraverse\n"
98 );
99         // 文件操作部分
100        printf("-----\n");
101        printf("          File Operation Functions\n");
102        printf("-----\n");
103        printf("          13. SaveList         14. LoadList\n");
104        // 附加功能
105        printf("-----\n");
106        printf("          Additional Functions\n");
107        printf("-----\n");
108        printf("          15. ReverseList     16. SortList\n");
109        printf("          17. DeleteNodeFromEnd\n");
110        // 只在多链表模式才显示多链表操作
111        if (mode == 2) {
112            printf("-----\n");
113            printf("          Multi-Lists Functions\n");
114            printf("-----\n");
115            printf("          18. AddList         21. LocateList\n"
116 );
117            printf("          19. RemoveList     22. PrintLists\n"
118 );
119            printf("          20. SwitchList     23. InitMultiLists\n");
120        }
121        printf("-----\n");
122        printf("          0. Return to Main Menu\n");
123        printf("-----\n");

```

```
120         printf("    Choose your operations [0~%d]:", mode ==
2 ? 21 : 16);
121         scanf("%d", &op);
122
123         if(op == 0)
124         {
125             printf("Returning to Main Menu...\n");
126             op = 1;
127             break;
128         }
129
130         if(mode == 2 && op != 18 && op != 20 && op != 22 &&
L_operated == NULL)
131         {
132             printf("No list exists! Please create a list
first.\n");
133             op = 1; // 重置操作码
134             getchar(); // 清除缓冲区中的换行符
135             getchar(); // 等待用户输入
136             continue;
137         }
138
139         switch(op)
140         {
141             case 1:
142                 if(InitList(L_operated) == OK)
143                     printf("InitList successfully!\n");
144                 else
145                     printf("InitList failed!\n");
146                 break;
147
148             case 2:
149                 if(DestroyList(L_operated) == OK)
150                     printf("DestroyList successfully!\n");
151                 if(mode == 2)
152                     RemoveList(&Lists, Lists.name[list_idx]);
153                 else
154                     printf("DestroyList failed!\n");
155                 break;
156
157             case 3:
158                 if(ClearList(L_operated) == OK)
159                     printf("ClearList successfully!\n");
160                 else
161                     printf("ClearList failed!\n");
162                 break;
163
164             case 4:
165                 if(ListEmpty(L_operated) == TRUE)
166                     printf("The list is empty!\n");
167                 else
168                     printf("The list is not empty!\n");
169                 break;
170
171             case 5:
```

```

172         if((length = ListLength(L_operated)) ==
INFEASIBLE)
173             printf("The list does not exist!\n");
174         else
175             printf("The length of the list is %d\n",
length);
176         break;
177
178         case 6:
179             int i;
180             ElemType e;
181             printf("Please input the index of the element
you want to get: ");
182             scanf("%d", &i);
183             if(GetElem(L_operated, i, &e) == OK)
184                 printf("The element at index %d is %d\n",
i, e);
185             else
186                 printf("GetElem failed!\n");
187             break;
188
189         case 7:
190             printf("Please input the element you want to
locate: ");
191             scanf("%d", &e);
192             if((i = LocateElem(L_operated, e)) != ERROR)
193                 printf("The element %d is located at
index %d\n", e, i);
194             else
195                 printf("LocateElem failed!\n");
196             break;
197
198         case 8:
199             printf("Please input the element you want to
find the prior of: ");
200             scanf("%d", &e);
201             ElemType priorElem;
202             if(PriorElem(L_operated, e, &priorElem) == OK
)
203                 printf("The prior element of %d is found!
\n", priorElem);
204             else
205                 printf("PriorElem failed!\n");
206             break;
207
208         case 9:
209             printf("Please input the element you want to
find the next of: ");
210             scanf("%d", &e);
211             ElemType nextElem;
212             if(NextElem(L_operated, e, &nextElem) == OK)
213                 printf("The next element of %d is found!\n",
nextElem);
214             else
215                 printf("NextElem failed!\n");

```

```
216         break;
217
218     case 10:
219         printf("Please input the index and the
element you want to insert: ");
220         scanf("%d %d", &i, &e);
221         if(ListInsert(L_operated, i, e) == OK)
222             printf("ListInsert successfully!\n");
223         else
224             printf("ListInsert failed!\n");
225         break;
226
227     case 11:
228         printf("Please input the index of the element
you want to delete: ");
229         scanf("%d", &i);
230         if(ListDelete(L_operated, i, &e) == OK)
231             printf("ListDelete successfully! Deleted
element: %d\n", e);
232         else
233             printf("ListDelete failed!\n");
234         break;
235
236     case 12:
237         if(printLists(L_operated) == OK)
238             printf("ListTraverse successfully!\n");
239         else
240             printf("ListTraverse failed!\n");
241         break;
242
243     case 13:
244         printf("Please input the file name to save
the list: ");
245         scanf("%s", fileName);
246         if(SaveList(L_operated, fileName) == OK)
247             printf("SaveList successfully!\n");
248         else
249             printf("SaveList failed!\n");
250         break;
251
252     case 14:
253         printf("Please input the file name to load
the list: ");
254         scanf("%s", fileName);
255         if(LoadList(L_operated, fileName) == OK)
256             printf("LoadList successfully!\n");
257         else
258             printf("LoadList failed!\n");
259         break;
260
261     case 15:
262         if(reverseList(L_operated) == OK)
263             printf("ReverseList successfully!\n");
264         else
265             printf("ReverseList failed!\n");
```


华中科技大学课程实验报告

```
266         break;
267
268     case 16:
269         if(SortList(L_operated) == OK)
270             printf("SortList successfully!\n");
271         else
272             printf("SortList failed!\n");
273         break;
274
275     case 17:
276         printf("Please input the value of n to delete
the nth node from the end: ");
277         scanf("%d", &n);
278         if((deletedElem = DeleteNodefromEnd(
L_operated, n)) != ERROR)
279             printf("DeleteNodeFromEnd successfully!
Deleted element: %d\n", deletedElem);
280         else
281             printf("DeleteNodeFromEnd failed!\n");
282         break;
283
284     case 18:
285         printf("Please input the name of the list you
want to add: ");
286         scanf("%s", Lists_name);
287         if(AddList(&Lists, Lists_name) == OK)
288             printf("AddList successfully!\n");
289         else
290             printf("AddList failed!\n");
291         break;
292
293     case 19:
294         printf("Please input the name of the list you
want to remove: ");
295         scanf("%s", Lists_name);
296         if(RemoveList(&Lists, Lists_name) == OK)
297             printf("RemoveList successfully!\n");
298         else
299             printf("RemoveList failed!\n");
300         break;
301
302     case 20:
303         printf("Please input the name of the list you
want to switch to: ");
304         scanf("%s", Lists_name);
305         if((list_idx = LocateList(Lists, Lists_name))
!= ERROR)
306             printf("SwitchList successfully!\n");
307         else
308             printf("SwitchList failed!\n");
309         break;
310
311     case 21:
312         printf("Please input the name of the list you
want to locate: ");
```

```
313         scanf("%s", Lists_name);
314         if((loc_idx = LocateList(Lists, Lists_name))
!= ERROR)
315             printf("LocateList successfully! List id:
%d\n", loc_idx + 1);
316         else
317             printf("LocateList failed!\n");
318         break;
319
320     case 22:
321         PrintLists(Lists);
322         break;
323
324     case 23:
325         if(InitMultiLists(&Lists) == OK)
326             printf("InitMultiLists successfully!\n");
327         else
328             printf("InitMultiLists failed!\n");
329         break;
330     }
331
332     printf("Press any key to continue...");
333     getchar(); // 清除缓冲区中的换行符
334     getchar(); // 等待用户输入
335
336 }
337
338 op = 1; // 重置操作码
339
340 }
341
342 return 0;
343 }
```

Listing 41 main.cpp

附录 C 基于二叉链表二叉树实现的源程序

本项目由以下源文件构筑：

- def.h - 定义了二叉树的相关数据结构和常量。
- BinaryTree.h - 定义了二叉树的相关操作函数。
- BiTree-Addons.h - 实现了二叉树的附加操作函数。
- BiTree-IO.h - 实现了二叉树的文件读写操作函数。
- BiTree-Multi.h - 实现了多二叉树的相关操作函数。
- BiTree-Traverse.h - 实现了二叉树的遍历操作函数。
- main.cpp - 主驱动程序

以下是各文件的详细代码：

def.h

```

1  #include "stdio.h"
2  #include "stdlib.h"
3  #include "string.h"
4
5  #define TRUE 1
6  #define FALSE 0
7  #define OK 1
8  #define ERROR 0
9  #define INFEASIBLE -1
10 #define OVERFLOW -2
11 #define INIT_TREE_NUM 20
12 #define INIT_TREE_SIZE 100
13
14 typedef int status;
15 typedef int KeyType;
16 typedef struct {
17     KeyType key;
18     char others[20];
19 } TElemType; //二叉树结点类型定义
20
21
22 typedef struct BiTNode{ //二叉链表结点的定义
23     TElemType data;
24     struct BiTNode *lchild,*rchild;
25 } BiTNode, *BiTree;
26
27 typedef struct BiTrees {
28     BiTree Trees[INIT_TREE_NUM]; // 二叉树数组
29     int num; // 二叉树数量
30     char name[INIT_TREE_NUM][20]; // 二叉树名称

```

```
31 } BiTrees;
32
33 //status SaveBiTree(BiTree T, char FileName[]);
34 //status LoadBiTree(BiTree &T, char FileName[]);
```

Listing 42 def.h

BinaryTree.h

```
1
2 // 全局变量，用于跟踪前序遍历序列的当前位置
3 int count_def = 0;
4
5 // 辅助函数：检查二叉树中是否存在相同关键字，但排除当前节点自身
6 status HasSameKey(BiTree T, KeyType key, BiTree currentNode) {
7     if (T == NULL) return FALSE;
8
9     // 检查当前节点的key是否匹配，但排除自身的比较
10    if (T != currentNode && T->data.key == key) return TRUE;
11
12    // 递归检查左右子树
13    if (HasSameKey(T->lchild, key, currentNode)) return TRUE;
14    if (HasSameKey(T->rchild, key, currentNode)) return TRUE;
15
16    return FALSE;
17 }
18
19 status CreateBiTree_Pre(BiTree *T, TElemType definition[])
20 //根据带空枝的二叉树先根遍历序列definition构造一棵二叉树，将根节
   点指针赋值给T并返回OK，
21 {
22     if(*T != NULL) return INFEASIBLE; // 如果树已经存在，返回错误
23
24     // 处理空节点标记（假设definition中的key为0表示空节点）
25     if (definition[count_def].key == 0) {
26         *T = NULL;
27         count_def++;
28         return OK;
29     }
30
31     // 创建临时树根指针，用于检查重复key
32     static BiTree root = NULL;
33
34     // 第一个节点时初始化根指针
35     if (count_def == 0) {
36         root = NULL;
37     }
38
39     // 分配新节点
40     *T = (BiTNode *)malloc(sizeof(BiTNode));
41     if (!(*T)) return ERROR; // 内存分配失败
42 }
```

```

43     (*T) -> lchild = NULL; // 初始化左子树
44     (*T) -> rchild = NULL; // 初始化右子树
45
46     // 设置节点数据
47     (*T)->data = definition[count_def];
48
49     // 检查树中是否已存在相同关键字
50     // 第一个节点时, root为空, 设置root
51     if (root == NULL) {
52         root = *T;
53     }
54     else if (HasSameKey(root, (*T)->data.key, *T)) { // 传入当前
节点T以便排除自身
55         free(*T); // 释放已分配的内存
56         *T = NULL;
57         return ERROR; // 存在相同关键字, 返回错误
58     }
59
60     // 递增count, 准备读取下一个节点
61     count_def++;
62
63     // 递归构建左子树
64     status leftStatus = CreateBiTree_Pre(&((*T)->lchild),
definition);
65     if (leftStatus == ERROR) return ERROR;
66
67     // 递归构建右子树
68     status rightStatus = CreateBiTree_Pre(&((*T)->rchild),
definition);
69     if (rightStatus == ERROR) return ERROR;
70
71     return OK;
72 }
73
74 typedef struct {
75     int pos;
76     TElemType data;
77 } DEF;
78
79 status CreateBiTree(BiTree *T, DEF definition[])
80 {
81     int i=0, j;
82     static BiTNode *p[100];
83     while (j=definition[i].pos)
84     {
85         p[j]=(BiTNode *)malloc(sizeof(BiTNode));
86         p[j]->data=definition[i].data;
87         p[j]->lchild=NULL;
88         p[j]->rchild=NULL;
89         if (j!=1)
90             if (j%2) p[j/2]->rchild=p[j];
91             else p[j/2]->lchild=p[j];
92         i++;
93     }
94     *T=p[1];

```

```
95     return OK;
96 }
97
98 status ClearBiTree(BiTree *T)
99 //将二叉树设置成空，并删除所有结点，释放结点空间
100 {
101     if(*T == NULL)
102         return OK;
103
104     status error_l = ClearBiTree(&((*T) -> lchild));
105     if(error_l == ERROR) return ERROR;
106     status error_r = ClearBiTree(&((*T)->rchild));
107     if(error_r == ERROR) return ERROR;
108
109     free(*T);
110     *T = NULL;
111
112     return OK;
113 }
114
115
116 bool BiTreeEmpty(BiTree T)
117 //判断二叉树是否为空
118 {
119     if(T == NULL)
120         return true;
121     else
122         return false;
123 }
124
125 int BiTreeDepth(BiTree T)
126 //求二叉树T的深度
127 {
128     // 空树深度为0
129     if(T == NULL)
130         return 0;
131
132     // 分别计算左右子树的深度
133     int depth_l = BiTreeDepth(T->lchild);
134     int depth_r = BiTreeDepth(T->rchild);
135
136     // 当前树的深度为较深子树的深度加1
137     return 1 + (depth_l > depth_r ? depth_l : depth_r);
138 }
139
140
141 BiTNode* LocateNode(BiTree T,KeyType e)
142 //查找结点
143 {
144     if(T == NULL)
145         return NULL;
146
147     BiTNode* loc_node = NULL;
148
149     if((T->data).key == e)
```

```
150     return T;
151 else
152 {
153     if(loc_node = LocateNode(T -> lchild,e))
154         return loc_node;
155     if(loc_node = LocateNode(T -> rchild,e))
156         return loc_node;
157 }
158
159 return loc_node;
160 }
161
162 BiTNode* GetParent(BiTree T, KeyType e)
163 //查找关键字为e的节点的父节点
164 {
165     // 空树或根节点没有父节点
166     if(T == NULL)
167         return NULL;
168
169     // 检查当前节点的左子节点
170     if(T->lchild != NULL && T->lchild->data.key == e)
171         return T; // 找到了, 返回父节点
172
173     // 检查当前节点的右子节点
174     if(T->rchild != NULL && T->rchild->data.key == e)
175         return T; // 找到了, 返回父节点
176
177     // 在左子树中递归查找
178     BiTNode* parent = NULL;
179     if(T->lchild != NULL) {
180         parent = GetParent(T->lchild, e);
181         if(parent != NULL)
182             return parent; // 在左子树中找到了父节点
183     }
184
185     // 在右子树中递归查找
186     if(T->rchild != NULL) {
187         parent = GetParent(T->rchild, e);
188         if(parent != NULL)
189             return parent; // 在右子树中找到了父节点
190     }
191
192     // 如果左右子树都没找到, 返回NULL
193     return NULL;
194 }
195
196 status Assign(BiTree *T, KeyType e, TElemType value)
197 //实现结点赋值。此题允许通过增加其它函数辅助实现本关任务
198 {
199     if (*T == NULL) return ERROR;
200
201     if (e != value.key) {
202         if (HasSameKey(*T, value.key, NULL)) {
203             return ERROR;
204         }
205     }
```

```

205     }
206
207     BiTNode *node = LocateNode(*T, e);
208
209     if (node == NULL) return ERROR;
210
211     node->data = value;
212
213     return OK;
214 }
215
216
217
218 BiTNode* GetSibling(BiTree T, KeyType e)
219 //实现获得兄弟结点
220 {
221     if (T == NULL || (T->lchild == NULL && T->rchild == NULL))
222         return NULL;
223
224     if (T->lchild != NULL && T->lchild->data.key == e)
225         return T->rchild;
226
227     if (T->rchild != NULL && T->rchild->data.key == e)
228         return T->lchild;
229
230     BiTNode* sibling = NULL;
231     if (T->lchild != NULL)
232         sibling = GetSibling(T->lchild, e);
233
234     if (sibling != NULL)
235         return sibling;
236
237     if (T->rchild != NULL)
238         sibling = GetSibling(T->rchild, e);
239
240     return sibling;
241 }
242
243 status InsertNode(BiTree *T, KeyType e, int LR, TElemType c)
244 {
245     // 特殊情况：插入作为根节点
246     if (LR == -1)
247     {
248         BiTNode* newNode = (BiTNode*)malloc(sizeof(BiTNode));
249         if (newNode == NULL) return ERROR;
250         newNode->data = c;
251         newNode->lchild = NULL;
252         newNode->rchild = *T; // 原树作为右子树
253         *T = newNode;
254         return OK;
255     }
256
257     if (HasSameKey(*T, c.key, NULL)) return ERROR;
258
259     BiTNode *basicNode = LocateNode(*T, e);

```



```
260     if(basicNode == NULL) return ERROR;
261
262     BiTNode* newNode = (BiTNode*)malloc(sizeof(BiTNode));
263     if(newNode == NULL) return ERROR;
264     newNode->data = c;
265     newNode->lchild = NULL;
266     newNode->rchild = NULL;
267
268     // 插入到左子树
269     if(LR == 0)
270     {
271         // 若原左子树存在, 则保存它
272         BiTNode* originalLeft = basicNode->lchild;
273
274         basicNode->lchild = newNode;
275
276         if(originalLeft != NULL) {
277             newNode->rchild = originalLeft;
278         }
279     }
280     // 插入到右子树
281     else if(LR == 1)
282     {
283         // 若原右子树存在, 则保存它
284         BiTNode* originalRight = basicNode->rchild;
285
286         basicNode->rchild = newNode;
287
288         if(originalRight != NULL) {
289             newNode->rchild = originalRight;
290         }
291     }
292     else
293         return ERROR; // 无效的LR值
294
295     return OK;
296 }
297
298 status DeleteNode(BiTree *T, KeyType e)
299 {
300     BiTNode *nodeToDelete = LocateNode(*T, e);
301     if(nodeToDelete == NULL) return ERROR;
302
303     BiTNode *parent = GetParent(*T, e);
304     if(parent == NULL) return ERROR; // 根节点没有父节点, 无法删除
305
306     status position = 0; // 0表示左子树, 1表示右子树
307     if(parent->rchild == nodeToDelete) position = 1;
308
309     BiTNode *lchild = nodeToDelete->lchild;
310     BiTNode *rchild = nodeToDelete->rchild;
311
312     if(lchild != NULL && rchild != NULL)
313     {
```

```
314     BiTNode *LC_most_right = lchild;
315     while(LC_most_right->rchild != NULL)
316         LC_most_right = LC_most_right->rchild;
317     LC_most_right->rchild = rchild;    // 将右子树连接到左子
树的最后节点
318
319     if(position == 0)
320         parent->lchild = lchild; // 将左子树连接到父节点
321     else
322         parent->rchild = lchild; // 将左子树连接到父节点
323 }
324 else if(lchild != NULL && rchild == NULL)
325 {
326     if(position == 0)
327         parent->lchild = lchild; // 将左子树连接到父节点
328     else
329         parent->rchild = lchild; // 将左子树连接到父节点
330 }
331 else if(lchild == NULL && rchild != NULL)
332 {
333     if(position == 0)
334         parent->lchild = rchild; // 将右子树连接到父节点
335     else
336         parent->rchild = rchild; // 将右子树连接到父节点
337 }
338 else
339 {
340     if(position == 0)
341         parent->lchild = NULL; // 删除左子树
342     else
343         parent->rchild = NULL; // 删除右子树
344 }
345
346 free(nodeToDelete);
347 nodeToDelete = NULL;
348
349 return OK;
350 }
351
352
353 status DeleteNode_HeadA(BiTree &T, KeyType e)
354 {
355     BiTNode *nodeToDelete = LocateNode(T, e);
356     if(nodeToDelete == NULL) return ERROR;
357
358     BiTNode *parent = GetParent(T, e);
359     // 处理根节点的特殊情况
360     if(parent == NULL) {
361         // 确认是根节点
362         if(nodeToDelete != T) return ERROR;
363
364         BiTNode *lchild = nodeToDelete->lchild;
365         BiTNode *rchild = nodeToDelete->rchild;
366
367         // 情况1: 根节点有两个子节点
```

```
368     if(lchild != NULL && rchild != NULL) {
369         // 找到左子树的最右节点
370         BiTNode *LC_most_right = lchild;
371         while(LC_most_right->rchild != NULL)
372             LC_most_right = LC_most_right->rchild;
373
374         // 将右子树接到左子树最右节点的右子树
375         LC_most_right->rchild = rchild;
376
377         // 用左子树替换根节点
378         T = lchild;
379     }
380     // 情况2: 根节点只有左子树
381     else if(lchild != NULL) {
382         T = lchild;
383     }
384     // 情况3: 根节点只有右子树
385     else if(rchild != NULL) {
386         T = rchild;
387     }
388     // 情况4: 根节点是叶子节点
389     else {
390         T = NULL;
391     }
392
393     free(nodeToDelete);
394     return OK;
395 }
396
397 status position = 0; // 0表示左子树, 1表示右子树
398 if(parent->rchild == nodeToDelete) position = 1;
399
400 BiTNode *lchild = nodeToDelete->lchild;
401 BiTNode *rchild = nodeToDelete->rchild;
402
403 if(lchild != NULL && rchild != NULL)
404 {
405     BiTNode *LC_most_right = lchild;
406     while(LC_most_right->rchild != NULL)
407         LC_most_right = LC_most_right->rchild;
408     LC_most_right->rchild = rchild; // 将右子树连接到左子
树的 最右节点
409
410     if(position == 0)
411         parent->lchild = lchild; // 将左子树连接到父节点
412     else
413         parent->rchild = lchild; // 将左子树连接到父节点
414
415 }
416
417 else if(lchild != NULL && rchild == NULL)
418 {
419     if(position == 0)
420         parent->lchild = lchild; // 将左子树连接到父节点
421     else
```

```
422         parent->rchild = lchild; // 将左子树连接到父节点
423     }
424     else if(lchild == NULL && rchild != NULL)
425     {
426         if(position == 0)
427             parent->lchild = rchild; // 将右子树连接到父节点
428         else
429             parent->rchild = rchild; // 将右子树连接到父节点
430     }
431     else
432     {
433         if(position == 0)
434             parent->lchild = NULL; // 删除左子树
435         else
436             parent->rchild = NULL; // 删除右子树
437     }
438
439     free(nodeToDelete);
440     nodeToDelete = NULL;
441
442     return OK;
443 }
444 }
```

Listing 43 BinaryTree.h

BiTree-Addons.h

```
1
2 //获取T的最大根节点到叶节点的key值和（一条路径）
3 int MaxPath(BiTree T)
4 {
5     if (T == NULL) return 0; // 空树返回0
6
7     // 计算左子树和右子树的最大路径和
8     int leftMax = MaxPath(T->lchild);
9     int rightMax = MaxPath(T->rchild);
10
11     // 返回当前节点的值加上左右子树的最大路径和
12     return T->data.key + (leftMax > rightMax ? leftMax : rightMax
13 );
14 }
15 BiTNode* LowestCommonAncestor(BiTree T, KeyType e1, KeyType e2)
16 {
17     if (T == NULL) return NULL; // 空树返回NULL
18
19     // 如果当前节点是e1或e2，则返回当前节点
20     if (T->data.key == e1 || T->data.key == e2) {
21         return T;
22     }
23 }
```

```
24 // 递归查找左子树和右子树
25 BiTNode* left = LowestCommonAncestor(T->lchild, e1, e2);
26 BiTNode* right = LowestCommonAncestor(T->rchild, e1, e2);
27
28 // 如果在左右子树中都找到了, 则当前节点是最近公共祖先
29 if (left != NULL && right != NULL) {
30     return T;
31 }
32
33 // 否则, 返回非NULL的子节点 (如果有的话)
34 return left != NULL ? left : right;
35 }
36
37 status InvertTree(BiTree *T)
38 {
39     if (*T == NULL) return OK; // 空树直接返回
40
41     // 交换左右子树
42     BiTree temp = (*T)->lchild;
43     (*T)->lchild = (*T)->rchild;
44     (*T)->rchild = temp;
45
46     // 递归反转左右子树
47     InvertTree(&(*T)->lchild);
48     InvertTree(&(*T)->rchild);
49
50     return OK;
51 }
```

Listing 44 BiTree-Addons.h

BiTree-IO.h

```
1 // 全局文件指针
2 FILE *g_outFile = NULL;
3
4 // 节点访问函数
5 void SaveNode(BiTree node)
6 {
7     if (node == NULL)
8         fprintf(g_outFile, "0 null ");
9     else
10         fprintf(g_outFile, "%d %s ", node->data.key, node->data.
11             others);
12 }
13
14 status SaveBiTree(BiTree T, char FileName[])
15 // 将二叉树的结点数据写入到文件FileName中
16 {
17     // 打开文件
18     FILE *fout = fopen(FileName, "w");
19     if (fout == NULL)
```

```
19     return ERROR;
20
21     // 设置全局文件指针
22     g_outFile = fout;
23
24     // 使用先序遍历保存节点
25     PreOrderTraverse(T, SaveNode);
26
27     // 关闭文件
28     fclose(fout);
29     return OK;
30 }
31
32 status LoadBiTree(BiTree *T, char FileName[]) // 修改为指针参数
33 {
34     FILE *fin = fopen(FileName, "r");
35     if(fin == NULL)
36         return ERROR;
37
38     TElemType definition[128]; // 假设最多128个节点
39     int i = 0;
40     do {
41         fscanf(fin, "%d%s", &definition[i].key, definition[i].others
42     );
43     } while (definition[i++].key != -1);
44     fclose(fin);
45
46     // 重新构建二叉树
47     *T = NULL; // 使用指针访问
48     CreateBiTree_Pre(T, definition); // 直接传递指针
49     //printf("%d %s\n", (*T)->lchild->data.key, (*T)->lchild->
50     data.others); // 输出根节点信息
51
52     return OK;
53 }
```

Listing 45 BiTree-IO.h

BiTree-Multi.h

```
1
2 status InitBiTrees(BiTrees &T)
3 {
4     T.num = 0; // 初始化二叉树数量为0
5     for (int i = 0; i < INIT_TREE_NUM; i++) {
6         T.Trees[i] = NULL; // 初始化每棵树为空
7         strcpy(T.name[i], ""); // 初始化名称为空字符串
8     }
9     return OK;
10 }
11
```

```
12 status ClearBiTrees(BiTrees &T)
13 {
14     for (int i = 0; i < T.num; i++) {
15         ClearBiTree(&T.Trees[i]); // 清空每棵树
16     }
17     T.num = 0; // 重置二叉树数量为0
18     return OK;
19 }
20
21 status AddBiTree(BiTrees &T, char name[])
22 {
23     if (T.num >= INIT_TREE_NUM) return ERROR;
24     T.Trees[T.num] = NULL;
25     strcpy(T.name[T.num], name);
26     T.num++;
27     return OK;
28 }
29
30 int LocateBiTree(BiTrees T, char name[])
31 {
32     for (int i = 0; i < T.num; i++) {
33         if (strcmp(T.name[i], name) == 0) return i;
34     }
35     return -1;
36 }
37
38 status DeleteBiTree(BiTrees &T, char name[])
39 {
40     int index = LocateBiTree(T, name);
41     if (index == -1) return ERROR;
42     ClearBiTree(&T.Trees[index]); // 清空树
43     for (int i = index; i < T.num - 1; i++) {
44         T.Trees[i] = T.Trees[i + 1]; // 移动树
45         strcpy(T.name[i], T.name[i + 1]); // 移动名称
46     }
47     T.num--; // 减少数量
48     return OK;
49 }
50
51 status PrintBiTrees(BiTrees T)
52 {
53     for (int i = 0; i < T.num; i++) {
54         printf("Tree %d: %s\n", i + 1, T.name[i]);
55     }
56     return OK;
57 }
```

Listing 46 BiTree-Multi.h

BiTree-Traverse.h

```
1 // 递归实现先序遍历
2
3 status PreOrderTraverse(BiTree T, void (*visit)(BiTree))
4 {
5     if(T != NULL)
6     {
7         visit(T); // 访问根节点
8         PreOrderTraverse(T->lchild, visit); // 先序遍历左子树
9         PreOrderTraverse(T->rchild, visit); // 先序遍历右子树
10    }
11    else
12        visit(NULL); // 访问空节点
13
14    return OK;
15 }
16
17 status InOrderTraverse(BiTree T, void (*visit)(BiTree))
18 {
19     if(T != NULL)
20     {
21         InOrderTraverse(T->lchild, visit); // 中序遍历左子树
22         visit(T); // 访问根节点
23         InOrderTraverse(T->rchild, visit); // 中序遍历右子树
24    }
25    else
26        visit(NULL); // 访问空节点
27    return OK;
28 }
29
30 status PostOrderTraverse(BiTree T, void (*visit)(BiTree))
31 {
32     if(T != NULL)
33     {
34         PostOrderTraverse(T->lchild, visit); // 后序遍历左子树
35         PostOrderTraverse(T->rchild, visit); // 后序遍历右子树
36         visit(T); // 访问根节点
37    }
38    else
39        visit(NULL); // 访问空节点
40    return OK;
41 }
42
43 status LevelOrderTraverse(BiTree T, void (*visit)(BiTree))
44 {
45     if(T == NULL) return OK; // 空树直接返回
46
47     // 使用数组实现简单队列
48     BiTree queue[128]; // 假设MAX_TREE_SIZE是预定义的足够大小
49     int front = 0; // 队列头指针
50     int rear = 0; // 队列尾指针
51
52     // 根节点入队
53     queue[rear++] = T;
54 }
```



```

55 // 当队列不为空时循环处理
56 while(front < rear)
57 {
58     // 出队一个节点并访问
59     BiTree node = queue[front++];
60     visit(node);
61
62     // 左子节点存在则入队
63     if(node->lchild != NULL)
64         queue[rear++] = node->lchild;
65
66     // 右子节点存在则入队
67     if(node->rchild != NULL)
68         queue[rear++] = node->rchild;
69 }
70
71 return OK;
72 }
73
74 status PreOrderTraverse_Stack(BiTree T, void (*visit)(BiTree))
75 {
76     if(T == NULL) return OK; // 空树直接返回
77
78     BiTree stack[128]; // 假设MAX_TREE_SIZE是预定义的足够大小
79     int top = -1; // 栈顶指针
80
81     // 根节点入栈
82     stack[++top] = T;
83
84     while(top >= 0)
85     {
86         // 出栈一个节点并访问
87         BiTree node = stack[top--];
88         visit(node);
89
90         // 右子节点先入栈，左子节点后入栈
91         if(node->rchild != NULL)
92             stack[++top] = node->rchild;
93         if(node->lchild != NULL)
94             stack[++top] = node->lchild;
95     }
96
97     return OK;
98 }
99
100 void ShowNode(BiTree T)
101 {
102     if(T != NULL)
103         printf("%d,%s ", T->data.key, T->data.others); // 显示节
104         点信息
105     else;
106
107     return;
108     // printf("null "); // 显示空节点
109 }

```

Listing 47 BiTree-Traversal.h

main.cpp

```

1
2 #include "def.h"
3 #include "BinaryTree.h"
4 #include "BiTree-Traversal.h"
5 #include "BiTree-IO.h"
6 #include "BiTree-Addons.h"
7 #include "BiTree-Multi.h"
8
9 extern int count_def; // 全局变量，用于跟踪前序遍历序列的当前位置
10
11 int main()
12 {
13     int mode = 1, op = 1, count = 0, temp = 0; // 操作码
14     BiTree *T_operated = NULL;
15     BiTree T_single = NULL; // 单棵二叉树指针
16     BiTrees Trees;
17     InitBiTrees(Trees); // 初始化多棵二叉树
18
19     DEF definition[100]; // 定义数组
20
21     char fileName[30];
22     char name[20];
23
24     int tree_idx = 0;
25     int tree_depth = 0;
26
27     KeyType key, old_key, sibling_key, insert_after_key,
28     delete_key, first_key, second_key;
29     BiTNode *node = NULL;
30     BiTNode *sibling = NULL;
31     BiTNode *lca = NULL;
32     TElemType new_value, new_node_value;
33     int LR;
34     int index;
35
36     int i, j, k, flag = 1;
37
38     while(mode)
39     {
40         system("cls");      printf("\n\n");
41         printf("Menu for Mode Selection \n");
42         printf("-----\n");
43         printf("1. Single Binary Tree \n");
44         printf("2. Multi Binary Tree \n");
45         printf("0. Exit \n");

```

```

45     printf("-----\n");
46     printf("        Choose the mode from 0 to 2: ");
47     scanf("%d",&mode);
48     T_operated = NULL; // 重置操作的链表指针
49
50     while(op)
51     {
52
53         count_def = 0; // 重置前序遍历序列的当前位置
54
55         if(mode == 2)
56         {
57             system("cls");    printf("\n\n");
58             printf("-----\n");
59             printf("        Multi-BiTrees Operation Mode:\n");
60             printf("-----\n");
61             if(Trees.name[tree_idx][0] == 0)
62             {
63                 T_operated = NULL;
64                 printf("        No trees\n");
65                 printf("-----\n");
66             }
67             else
68             {
69                 T_operated = &(Trees.Trees[tree_idx]);
70                 printf("        Current list: %s",
Trees.name[tree_idx]);
71                 printf("        List id: %d", tree_idx + 1);
72                 if(Trees.Trees[tree_idx] == NULL)
73                     printf("        (Uinit)");
74                 printf("\n-----\n");
75             }
76         }
77         else if(mode == 1)
78         {
79             T_operated = &T_single;
80             system("cls");    printf("\n\n");
81             printf("-----\n");
82             printf("        Single-Tree Operation Mode:\n");
83             printf("-----\n");
84         }
85         else if(mode == 0)
86         {
87             printf("Exiting...\n");
88             break;

```

```

89         }
90         else
91         {
92             printf("Invalid mode!\n");
93             continue;
94         }
95
96         printf("                Menu for Operation Selection \n");
97         printf("-----\n");
98         printf("                1. CreateBiTree                8.\n");
99         GetSibling\n");
100        printf("                2. DestroyBiTree               9.\n");
101        InsertNode\n");
102        printf("                3. ClearBiTree                 10.\n");
103        DeleteNode\n");
104        printf("                4. BiTreeEmpty                 11.\n");
105        PreOrderTrav\n");
106        printf("                5. BiTreeDepth                 12.\n");
107        InOrderTrav\n");
108        printf("                6. LocateNode                  13.\n");
109        PostOrderTrav\n");
110        printf("                7. Assign                      14.\n");
111        LevelOrderTrav\n");
112        printf("-----\n");
113        printf("                File Operation Menu \n");
114        printf("-----\n");
115        printf("                15. SaveBiTree                 16.\n");
116        LoadBiTree\n");
117        printf("-----\n");
118        printf("                Additional Functions Menu \n");
119        printf("-----\n");
120        printf("                17. MaxPathSum 19. InvertTree\n");
121        printf("                18. LowestCommonAncestor\n");
122        printf("-----\n");
123
124        if(mode == 2)
125        {
126            printf("                Multi-BiTrees Operation\n");
127            Selection \n");
128            printf("-----\n");
129            printf("                20. AddBiTree                 23.\n");
130            DeleteBiTree\n");
131            printf("                21. LocateBiTree              24.\n");
132            PrintBiTrees\n");
133            printf("                22. ClearBiTrees              25.\n");
134            InitBiTrees \n");
135            printf("-----\n");
136            printf("-----\n");

```

```

124         }
125
126         printf("                0.Exit\n");
127         printf("-----\n\n");
128         printf("        Choose your operations [0~%d]: ", mode ==
2 ? 25 : 14);
129
130         scanf("%d", &op);
131         if(op == 0)
132         {
133             printf("Returning to Main Menu...\n");
134             op = 1;
135             break;
136         }
137
138         if(mode == 2 && op != 20 && op != 22 && op != 24 &&
T_operated == NULL)
139         {
140             printf("No trees exist! Please create a tree
first.\n");
141             op = 1; // 重置操作码
142             getchar(); // 清除缓冲区中的换行符
143             getchar(); // 等待用户输入
144             continue;
145         }
146
147         switch(op)
148         {
149             case 1:
150                 printf("Please input the definition of
the tree (end with 0): ");
151                 i = 0;
152                 do {
153                     scanf("%d%d%s",&definition[i].pos,&
definition[i].data.key,definition[i].data.others);
154                 } while (definition[i++].pos);
155
156                 while(definition[j].pos) // 去掉最后一个0
157                 {
158                     printf("definition[%d]: %d %d %s\n",
j, definition[j].pos, definition[j].data.key, definition[j].
data.others);
159                     j++;
160                 }
161
162                 if(!flag)
163                 {
164                     printf("Duplicate key found! Please
re-enter the definition.\n");
165                     flag = 1; // 重置标志
166                     break; // 重新输入定义
167                 }
168
169                 if(CreateBiTree(T_operated, definition) =

```

```

170         = OK)
171             printf("Create Tree Successfully!\n")
172     ; // 创建二叉树
173         else
174             printf("Create Tree Failed!\n"); //
175     创建失败
176         break;
177     case 2:
178         if(ClearBiTree(T_operated) == OK)
179             printf("Destroy Tree Successfully!\n")
180     ); // 销毁二叉树
181         else
182             printf("Destroy Tree Failed!\n"); //
183     销毁失败
184         break;
185     case 3:
186         if(ClearBiTree(T_operated) == OK)
187             printf("Clear Tree Successfully!\n");
188     // 清空二叉树
189         else
190             printf("Clear Tree Failed!\n"); // 清
191     空失败
192         break;
193     case 4:
194         if(BiTreeEmpty(*T_operated) == TRUE)
195             printf("The tree is empty!\n"); // 二
196     叉树为空
197         else
198             printf("The tree is not empty!\n");
199     // 二叉树不为空
200         break;
201     case 5:
202         if((tree_depth = BiTreeDepth(*T_operated)
203     ) == INFEASIBLE)
204             printf("The tree does not exist!\n");
205     // 二叉树不存在
206         else
207             printf("The depth of the tree is: %d\
208     n", tree_depth); // 输出二叉树深度
209         break;
210     case 6:
211         printf("Please input the key of the node
212     you want to locate: ");
213         scanf("%d", &key);
214         node = LocateNode(*T_operated, key); //
215     查找节点
216         if(node != NULL)
217             printf("Node found! Node: %d,%s\n",
218     node->data.key,node->data.others); // 节点存在
219         else

```

```

210         printf("Node not found!\n"); // 节点
    不存在
211         break;
212
213     case 7:
214         printf("Please input the key of the node
    you want to assign: ");
215         scanf("%d", &old_key);
216         printf("Please input the new value (key
    and others): ");
217         scanf("%d %s", &new_value.key, new_value.
    others);
218         if(Assign(T_operated, old_key, new_value)
    == OK)
219             printf("Assign Successfully!\n"); //
    赋值成功
220         else
221             printf("Assign Failed!\n"); // 赋值失
    败
222         break;
223
224     case 8:
225         printf("Please input the key of the node
    you want to get its sibling: ");
226         scanf("%d", &sibling_key);
227         sibling = GetSibling(*T_operated,
    sibling_key); // 获取兄弟节点
228         if(sibling != NULL)
229             printf("Sibling found! Sibling: %d,%s
    \n", sibling->data.key, sibling->data.others); // 兄弟节点存在
230         else
231             printf("Sibling not found!\n"); // 兄
    弟节点不存在
232         break;
233
234     case 9:
235         printf("Please input the key of the node
    you want to insert after: ");
236         scanf("%d", &insert_after_key);
237         printf("Please input 0 for left child or
    1 for right child: ");
238         scanf("%d", &LR);
239         printf("Please input the new node value (
    key and others): ");
240         scanf("%d %s", &new_node_value.key,
    new_node_value.others);
241         if(InsertNode(T_operated,
    insert_after_key, LR, new_node_value) == OK)
242             printf("Insert Node Successfully!\n")
    ; // 插入节点成功
243         else
244             printf("Insert Node Failed!\n"); //
    插入节点失败
245         break;
246

```

```

247         case 10:
248             printf("Please input the key of the node
you want to delete: ");
249             scanf("%d", &delete_key);
250             if(DeleteNode(T_operated, delete_key) ==
OK)
251                 printf("Delete Node Successfully!\n")
; // 删除节点成功
252             else
253                 printf("Delete Node Failed!\n"); //
删除节点失败
254             break;
255
256         case 11:
257             PreOrderTraverse(*T_operated, ShowNode);
// 前序遍历
258             break;
259
260         case 12:
261             InOrderTraverse(*T_operated, ShowNode); //
中序遍历
262             break;
263
264         case 13:
265             PostOrderTraverse(*T_operated, ShowNode);
// 后序遍历
266             break;
267
268         case 14:
269             LevelOrderTraverse(*T_operated, ShowNode);
// 层序遍历
270             break;
271
272         case 15:
273             printf("Please input the file name to
save the tree: ");
274             scanf("%s", fileName);
275             if(SaveBiTree(*T_operated, fileName) ==
OK)
276                 printf("SaveBiTree Successfully!\n");
// 保存二叉树成功
277             else
278                 printf("SaveBiTree Failed!\n"); // 保
存二叉树失败
279             break;
280
281         case 16:
282             printf("Please input the file name to
load the tree: ");
283             scanf("%s", fileName);
284             if(LoadBiTree(T_operated, fileName) == OK
)
285                 printf("LoadBiTree Successfully!\n");
// 加载二叉树成功
286             else

```


华中科技大学课程实验报告

```
287         printf("LoadBiTree Failed!\n"); // 加载二叉树失败
288         break;
289
290     case 17:
291         printf("MaxPathSum: %d\n", MaxPath(*T_operated)); // 最大路径和
292         printf("MaxPathSum Successfully!\n");
293         // 最大路径和成功
294         break;
295
296     case 18:
297         printf("Please input the key of the first
298         node: ");
299         scanf("%d", &first_key);
300         printf("Please input the key of the
301         second node: ");
302         scanf("%d", &second_key);
303         lca = LowestCommonAncestor(*T_operated,
304         first_key, second_key); // 最近公共祖先
305         if(lca != NULL)
306             printf("Lowest Common Ancestor found!
307             LCA: %d,%s\n", lca->data.key, lca->data.others); // 最近公共祖先存在
308         else
309             printf("Lowest Common Ancestor not
310             found!\n"); // 最近公共祖先不存在
311         break;
312
313     case 19:
314         if(InvertTree(T_operated) == OK)
315             printf("Invert Tree Successfully!\n");
316         // 反转二叉树成功
317         else
318             printf("Invert Tree Failed!\n"); // 反转二叉树失败
319         break;
320
321     case 20:
322         printf("Please input the name of the new
323         tree: ");
324         scanf("%s", name);
325         if(AddBiTree(Trees, name) == OK)
326             printf("AddBiTree Successfully!\n");
327         // 添加二叉树成功
328         else
329             printf("AddBiTree Failed!\n"); // 添加二叉树失败
330         break;
331
332     case 21:
333         printf("Please input the name of the tree
334         you want to locate: ");
335         scanf("%s", name);
336         index = LocateBiTree(Trees, name); // 查
```

```

找二叉树
327         if(index != -1)
328         {
329             printf("Tree found! Tree: %s\n",
Trees.name[index]); // 二叉树存在
330             tree_idx = index; // 更新当前树索引
331         }
332         else
333             printf("Tree not found!\n"); // 二叉
树不存在
334             break;
335
336         case 22:
337             if(ClearBiTrees(Trees) == OK)
338                 printf("Clear BiTrees Successfully!\n
"); // 清空多棵二叉树成功
339             else
340                 printf("Clear BiTrees Failed!\n"); //
清空多棵二叉树失败
341             break;
342
343         case 23:
344             printf("Please input the name of the tree
you want to delete: ");
345             scanf("%s", name);
346             if>DeleteBiTree(Trees, name) == OK)
347                 printf("Delete BiTree Successfully!\n
"); // 删除二叉树成功
348             else
349                 printf("Delete BiTree Failed!\n"); //
删除二叉树失败
350             break;
351
352         case 24:
353             if(PrintBiTrees(Trees) == OK)
354                 printf("Print BiTrees Successfully!\n
"); // 打印多棵二叉树成功
355             else
356                 printf("Print BiTrees Failed!\n"); //
打印多棵二叉树失败
357             break;
358
359         case 25:
360             if(InitBiTrees(Trees) == OK)
361                 printf("Init BiTrees Successfully!\n
"); // 初始化多棵二叉树成功
362             else
363                 printf("Init BiTrees Failed!\n"); //
初始化多棵二叉树失败
364             break;
365
366         default:
367             printf("Invalid operation!\n"); // 无效操
作
368             break;

```

```
369         }
370     }
371
372     printf("Press any key to continue...");
373     getchar(); // 清除缓冲区中的换行符
374     getchar(); // 等待用户输入
375 }
376
377 op = 1; // 重置操作码
378
379 }
380
381 return 0; // 返回0表示程序正常结束
382
383 }
```

Listing 48 main.cpp

附录 D 基于邻接表图实现的源程序

本项目由以下源文件构筑

- def.h
- Graph.h
- Graph-IO.h
- Graph-Addons.h
- Graph-Multi.h
- main.cpp

以下是各文件的详细代码

def.h

```

1  #include "stdio.h"
2  #include "stdlib.h"
3  #include "string.h"
4  #define TRUE 1
5  #define FALSE 0
6  #define OK 1
7  #define ERROR 0
8  #define INFEASIBLE -1
9  #define OVERFLOW -2
10 #define MAX_VERTEX_NUM 20
11 #define MAX_ARC_NUM 100
12 typedef int status;
13 typedef int KeyType;
14 typedef enum {DG,DN,UDG,UDN} GraphKind;
15 typedef struct {
16     KeyType key;
17     char others[20];
18 } VertexType; //顶点类型定义
19
20
21 typedef struct ArcNode {           //表结点类型定义
22     int adjvex;                    //顶点位置编号
23     struct ArcNode *nextarc;       //下一个表结点指针
24 } ArcNode;
25 typedef struct VNode{              //头结点及其数组类型定义
26     VertexType data;               //顶点信息
27     ArcNode *firstarc;             //指向第一条弧
28 } VNode, AdjList[MAX_VERTEX_NUM];
29 typedef struct { //邻接表的类型定义
30     AdjList vertices;              //头结点数组
31     int vexnum, arcnum;            //顶点数、弧数

```

```
32     GraphKind  kind;           //图的类型
33     } ALGraph;
34
35     typedef struct {
36         ALGraph Ggroup[MAX_VERTEX_NUM];
37         int Gnum;
38         char name[MAX_VERTEX_NUM][20];
39     } GraphGroup;
```

Listing 49 def.h

Graph.h

```
1
2 int GetLocation(ALGraph &G, KeyType key)
3 {
4     int i;
5
6     for(i = 0; i < G.vexnum; i++)
7     {
8         if(G.vertices[i].data.key == key)
9             return i;
10    }
11
12    return -1;
13 }
14
15 status CreateGraph(ALGraph &G, VertexType V[], KeyType VR[][2])
16 {
17     int i, j;
18     ArcNode *p1, *p2;
19
20     for(i = 0; V[i].key != -1; i++);
21
22     G.vexnum = i;
23     G.arcnum = 0;
24
25     for(i = 0; i < G.vexnum; i++)
26     {
27         G.vertices[i].data = V[i];
28         G.vertices[i].firstarc = NULL;
29     }
30
31     if(i < G.vexnum || i == 0)
32         return ERROR;
33
34     for(i = 0; i < G.vexnum; i++)
35     {
36         for(j = i + 1; j < G.vexnum; j++)
37         {
38             if(V[i].key == V[j].key)
39                 return ERROR;
```

```

40     }
41 }
42
43 for(j = 0; VR[j][0] != -1; j++)
44 {
45     int loc1, loc2;
46     loc1 = GetLocation(G, VR[j][0]);
47     loc2 = GetLocation(G, VR[j][1]);
48
49     if(loc1 == -1 || loc2 == -1)
50         return ERROR;
51
52     p1 = (ArcNode *)malloc(sizeof(ArcNode));
53     p1 -> adjvex = loc2;
54     p2 = (ArcNode *)malloc(sizeof(ArcNode));
55     p2 -> adjvex = loc1;
56
57     ArcNode *temp = G.vertices[loc1].firstarc;
58     G.vertices[loc1].firstarc = p1;
59     p1 -> nextarc = temp;
60     temp = G.vertices[loc2].firstarc;
61     G.vertices[loc2].firstarc = p2;
62     p2 -> nextarc = temp;
63     G.arcnum++;
64
65 }
66
67 G.kind = UDN;
68 return OK;
69
70 }
71
72 status DestroyGraph(ALGraph &G)
73 {
74     int i;
75     ArcNode *p1, *p2;
76
77     for(i = 0; i < G.vexnum; i++)
78     {
79         p1 = G.vertices[i].firstarc;
80         while(p1 != NULL)
81         {
82             p2 = p1;
83             p1 = p1 -> nextarc;
84             free(p2);
85         }
86     }
87
88     G.vexnum = 0;
89     G.arcnum = 0;
90
91     return OK;
92 }
93
94 status PutVex(ALGraph &G, KeyType u, VertexType value)

```

```

95 //根据u在图G中查找顶点, 查找成功将该顶点值修改成value, 返回OK;
96 //如果查找失败或关键字不唯一, 返回ERROR
97 {
98     int i = GetLocation(G, u);
99     if(i == -1) return ERROR;
100
101     for(int j = 0; j < G.vexnum; j++)
102     {
103         if(G.vertices[j].data.key == value.key && i != j)
104             return ERROR;
105     }
106
107     G.vertices[i].data = value;
108     return OK;
109 }
110
111 int FirstAdjVex(ALGraph G,KeyType u)
112 //根据u在图G中查找顶点, 查找成功返回顶点u的第一邻接顶点位序, 否则
    返回-1;
113 {
114     int i = GetLocation(G, u);
115     if(i == -1) return -1;
116
117     ArcNode *p = G.vertices[i].firstarc;
118     if(p == NULL) return -1;
119
120     return p -> adjvex;
121 }
122
123
124 int NextAdjVex(ALGraph G,KeyType v,KeyType w)
125 //v对应G的一个顶点,w对应v的邻接顶点; 操作结果是返回v的(相对于w)
    下一个邻接顶点的位序; 如果w是最后一个邻接顶点, 或v、w对应顶点
    不存在, 则返回-1。
126 {
127     int i = GetLocation(G, v);
128     int j = GetLocation(G, w);
129     if(i == -1) return -1;
130
131     ArcNode *p = G.vertices[i].firstarc;
132     while(p != NULL)
133     {
134         if(p -> adjvex == j)
135             break;
136         p = p -> nextarc;
137     }
138
139     if(p == NULL || p -> nextarc == NULL) return -1;
140
141     return p -> nextarc -> adjvex;
142 }
143
144 status InsertVex(ALGraph &G,VertexType v)
145 //在图G中插入顶点v, 成功返回OK, 否则返回ERROR
146 {

```

```

147     if(G.vexnum >= MAX_VERTEX_NUM) return ERROR;
148
149     for(int i = 0; i < G.vexnum; i++)
150     {
151         if(G.vertices[i].data.key == v.key)
152             return ERROR;
153     }
154
155     G.vertices[G.vexnum].data = v;
156     G.vertices[G.vexnum].firstarc = NULL;
157     G.vexnum++;
158
159     return OK;
160
161 }
162
163 status DeleteVex(ALGraph &G, KeyType u)
164 {
165     int i = GetLocation(G, u);
166     if(i == -1) return ERROR;
167
168     // 计数待删除的边数
169     int edgeCount = 0;
170
171     // 释放顶点的出边
172     ArcNode *p = G.vertices[i].firstarc;
173     while(p != NULL)
174     {
175         ArcNode *temp = p;
176         p = p->nextarc;
177         free(temp);
178         edgeCount++; // 每释放一条边，计数加1
179     }
180
181     // 处理其他顶点的入边和对应的出边
182     for(int j = 0; j < G.vexnum; j++)
183     {
184         if(j == i) continue; // 跳过被删除的顶点
185
186         ArcNode *p1 = G.vertices[j].firstarc;
187         ArcNode *p2 = NULL;
188
189         while(p1 != NULL)
190         {
191             if(p1->adjvex == i) // 找到指向要删除顶点的边
192             {
193                 // 删除这条边
194                 if(p2 == NULL) // 如果是第一条边
195                 {
196                     G.vertices[j].firstarc = p1->nextarc;
197                     free(p1);
198                     p1 = G.vertices[j].firstarc;
199                 }
200                 else // 如果不是第一条边
201                 {

```



```

202         p2->nextarc = p1->nextarc;
203         free(p1);
204         p1 = p2->nextarc;
205     }
206     edgeCount++; // 每删除一条边, 计数加1
207 }
208 else
209 {
210     // 调整大于i的adjvex值
211     if(p1->adjvex > i)
212         p1->adjvex--;
213
214     p2 = p1;
215     p1 = p1->nextarc;
216 }
217 }
218 }
219
220 // 移动顶点数组
221 for(int j = i; j < G.vexnum - 1; j++)
222 {
223     G.vertices[j] = G.vertices[j + 1];
224 }
225
226 // 更新顶点数和边数
227 G.vexnum--;
228 G.arcnum -= edgeCount / 2; // 因为是无向图, 每条边计算了两次
229
230 if(G.vexnum == 0) return ERROR;
231
232 // printf("Finished\n");
233
234 return OK;
235 }
236
237 status InsertArc(ALGraph &G, KeyType u, KeyType v)
238 {
239     int loc1 = GetLocation(G, u);
240     int loc2 = GetLocation(G, v);
241     if(loc1 == -1 || loc2 == -1) return ERROR;
242
243     //check if the edge already exists
244     ArcNode *p = G.vertices[loc1].firstarc;
245     while(p != NULL)
246     {
247         if(p->adjvex == loc2) return ERROR;
248         p = p->nextarc;
249     }
250
251     ArcNode *p1 = (ArcNode *)malloc(sizeof(ArcNode));
252     p1->adjvex = loc2;
253     p1->nextarc = G.vertices[loc1].firstarc;
254     G.vertices[loc1].firstarc = p1;
255
256     ArcNode *p2 = (ArcNode *)malloc(sizeof(ArcNode));

```

```
257     p2->adjvex = loc1;
258     p2->nextarc = G.vertices[loc2].firstarc;
259     G.vertices[loc2].firstarc = p2;
260
261     G.arcnum++;
262
263     return OK;
264 }
265
266 status DeleteArc(ALGraph &G,KeyType v,KeyType w)
267 {
268     int loc1 = GetLocation(G, v);
269     int loc2 = GetLocation(G, w);
270     if(loc1 == -1 || loc2 == -1) return ERROR;
271
272     int flag = 0;
273
274     ArcNode *p = G.vertices[loc1].firstarc;
275     ArcNode *p2 = NULL;
276
277     while(p != NULL)
278     {
279         if(p->adjvex == loc2)
280         {
281             if(p2 == NULL) // 如果是第一条边
282             {
283                 G.vertices[loc1].firstarc = p->nextarc;
284                 free(p);
285                 p = G.vertices[loc1].firstarc;
286             }
287             else // 如果不是第一条边
288             {
289                 p2->nextarc = p->nextarc;
290                 free(p);
291                 p = p2->nextarc;
292             }
293
294             flag = 1;
295         }
296         else
297         {
298             p2 = p;
299             p = p->nextarc;
300         }
301     }
302
303     if(!flag) return ERROR;
304
305     // 删除第二个顶点的出边
306     p = G.vertices[loc2].firstarc;
307     p2 = NULL;
308
309     while(p != NULL)
310     {
311
```

```

312     if(p->adjvex == loc1)
313     {
314         if(p2 == NULL) // 如果是第一条边
315         {
316             G.vertices[loc2].firstarc = p->nextarc;
317             free(p);
318             p = G.vertices[loc2].firstarc;
319         }
320         else // 如果不是第一条边
321         {
322             p2->nextarc = p->nextarc;
323             free(p);
324             p = p2->nextarc;
325         }
326     }
327     else
328     {
329         p2 = p;
330         p = p->nextarc;
331     }
332 }
333
334 G.arcnum--;
335
336 return OK;
337 }
338
339 // 辅助函数：递归进行深度优先搜索
340 void DFS(ALGraph &G, int v, int visited[], void (*visit)(
VertexType)) {
341     // 访问当前顶点
342     if(visit != NULL)
343         visit(G.vertices[v].data);
344     visited[v] = true;
345
346     // 遍历当前顶点的所有邻接顶点
347     ArcNode *p = G.vertices[v].firstarc;
348     while (p != NULL) {
349         // 如果邻接顶点未被访问，则递归访问它
350         if (!visited[p->adjvex]) {
351             DFS(G, p->adjvex, visited, visit);
352         }
353         p = p->nextarc;
354     }
355 }
356
357 status DFSTraverse(ALGraph &G, void (*visit)(VertexType))
358 // 对图G进行深度优先搜索遍历，依次对图中的每一个顶点使用函数visit
// 访问一次，且仅访问一次
359 {
360     if (G.vexnum == 0) return ERROR; // 如果图为空，返回错误
361
362     // 初始化访问标记数组
363     int visited[MAX_VERTEX_NUM];
364     for (int i = 0; i < G.vexnum; i++) {

```

```

365     visited[i] = false;
366 }
367
368 // 从每个未访问的顶点开始进行深度优先搜索
369 for (int i = 0; i < G.vexnum; i++) {
370     if (!visited[i]) {
371         DFS(G, i, visited, visit);
372     }
373 }
374
375 return OK;
376 }
377
378 status BFSTraverse(ALGraph &G, void (*visit)(VertexType))
379 //对图G进行广度优先搜索遍历, 依次对图中的每一个顶点使用函数visit
    访问一次, 且仅访问一次
380 {
381     if (G.vexnum == 0) return ERROR; // 如果图为空, 返回错误
382
383     // 初始化访问标记数组
384     bool visited[MAX_VERTEX_NUM];
385     for (int i = 0; i < G.vexnum; i++) {
386         visited[i] = false;
387     }
388
389     // 创建队列用于BFS
390     int queue[MAX_VERTEX_NUM];
391     int front = 0;
392     int rear = 0;
393
394     // 从每个未访问的顶点开始进行广度优先搜索
395     for (int i = 0; i < G.vexnum; i++) {
396         if (!visited[i]) {
397             // 访问当前顶点
398             visit(G.vertices[i].data);
399             visited[i] = true;
400
401             // 将当前顶点入队
402             queue[rear++] = i;
403
404             // 队列非空时循环处理
405             while (front < rear) {
406                 // 出队一个顶点
407                 int v = queue[front++];
408
409                 // 访问该顶点的所有未访问邻接点
410                 ArcNode *p = G.vertices[v].firstarc;
411                 while (p != NULL) {
412                     int w = p->adjvex;
413                     if (!visited[w]) {
414                         visit(G.vertices[w].data);
415                         visited[w] = true;
416                         queue[rear++] = w; // 新发现的顶点入队
417                     }
418                     p = p->nextarc;

```

```
419         }
420     }
421 }
422 }
423
424     return OK;
425 }
426
427 void PrintVertex(VertexType v)
428 {
429     printf("Key: %d, Others: %s\n", v.key, v.others);
430 }
```

Listing 50 Graph.h

Graph-IO.h

```
1     status SaveGraph(ALGraph G, char FileName[])
2 //将图的数据写入到文件FileName中
3 {
4     FILE *fp;
5     int i, j, edgeCount = 0;
6     ArcNode *p;
7
8     // 创建边的数组用于排序
9     typedef struct {
10         KeyType v1;
11         KeyType v2;
12     } Edge;
13
14     Edge edges[MAX_ARC_NUM];
15
16     // 打开文件, 如果失败则返回ERROR
17     if ((fp = fopen(FileName, "w")) == NULL) {
18         return ERROR;
19     }
20
21     // 写入图的基本信息: 类型、顶点数和边数
22     fprintf(fp, "%d %d %d\n", G.kind, G.vexnum, G.arcnum);
23
24     // 写入所有顶点的信息 (按照数组顺序)
25     for (i = 0; i < G.vexnum; i++) {
26         fprintf(fp, "%d %s\n", G.vertices[i].data.key, G.vertices[i].data.others);
27     }
28
29     // 收集所有边的信息
30     for (i = 0; i < G.vexnum; i++) {
31         p = G.vertices[i].firstarc;
32         while (p != NULL) {
33             // 对于无向图, 只收集一次边 (只有当i<p->adjvex时)
34             if (G.kind == DG || G.kind == DN || i < p->adjvex) {
```

```

35         edges[edgeCount].v1 = G.vertices[i].data.key;
36         edges[edgeCount].v2 = G.vertices[p->adjvex].data.
key;
37         edgeCount++;
38     }
39     p = p->nextarc;
40 }
41 }
42
43 // 对边进行冒泡排序（先按v1排序，v1相同时按v2排序）
44 for (i = 0; i < edgeCount - 1; i++) {
45     for (j = 0; j < edgeCount - i - 1; j++) {
46         if (edges[j].v1 > edges[j + 1].v1 ||
47             (edges[j].v1 == edges[j + 1].v1 && edges[j].v2 >
edges[j + 1].v2)) {
48             Edge temp = edges[j];
49             edges[j] = edges[j + 1];
50             edges[j + 1] = temp;
51         }
52     }
53 }
54
55 // 写入排序后的边信息
56 for (i = 0; i < edgeCount; i++) {
57     fprintf(fp, "%d %d\n", edges[i].v1, edges[i].v2);
58 }
59
60 fclose(fp);
61 return OK;
62 }
63
64 status LoadGraph(ALGraph &G, char FileName[])
65 //读入文件FileName的图数据，创建图的邻接表
66 {
67     FILE *fp;
68     int i, arcnum;
69     KeyType v1, v2;
70     VertexType vertices[MAX_VERTEX_NUM];
71     KeyType VR[MAX_ARC_NUM][2];
72     char others[20];
73
74     // 打开文件，如果失败则返回ERROR
75     if ((fp = fopen(FileName, "r")) == NULL) {
76         return ERROR;
77     }
78
79     // 读取图的基本信息：类型、顶点数和边数
80     if (fscanf(fp, "%d %d %d", &G.kind, &G.vexnum, &arcnum) != 3)
81     {
82         fclose(fp);
83         return ERROR;
84     }
85
86     // 读取顶点信息
87     for (i = 0; i < G.vexnum; i++) {

```

```
87     if (fscanf(fp, "%d %s", &vertices[i].key, others) != 2) {
88         fclose(fp);
89         return ERROR;
90     }
91     strcpy(vertices[i].others, others);
92 }
93
94 // 设置结束标记
95 vertices[i].key = -1;
96
97 // 读取边信息
98 i = 0;
99 while (fscanf(fp, "%d %d", &v1, &v2) == 2 && i < MAX_ARC_NUM)
100 {
101     VR[i][0] = v1;
102     VR[i][1] = v2;
103     i++;
104 }
105
106 // 设置边列表的结束标记
107 VR[i][0] = -1;
108 VR[i][1] = -1;
109
110 fclose(fp);
111
112 // 使用CreateGraph函数创建图
113 if (CreateGraph(G, vertices, VR) != OK) {
114     return ERROR;
115 }
116
117 return OK;
118 }
```

Listing 51 Graph-IO.h

Graph-Addons.h

```
1 status BFSTraverse(ALGraph &G, void (*visit)(VertexType));
2 void DFS(ALGraph &G, int v, bool visited[], void (*visit)(
3     VertexType));
4
5 //Get All vertices whose distance from the certain vertex is less
6 //than k
7 VertexType *VerticesSetLessThanK(ALGraph G, KeyType v, int k)
8 {
9     int i;
10     int visited[MAX_VERTEX_NUM] = {0};
11     int distance[MAX_VERTEX_NUM];
12     VertexType *result = (VertexType *)malloc(G.vexnum * sizeof(
13     VertexType));
14     int count = 0;
```

```

13     for(int i = 0; i < G.vexnum; i++)
14     {
15         result[i].key = -1; // 初始化结果数组
16     }
17
18     // 初始化访问标记和距离数组
19     for (i = 0; i < G.vexnum; i++) {
20         visited[i] = 0;
21         distance[i] = -1;
22     }
23
24     // 查找起始顶点的位置
25     int startIndex = GetLocation(G, v);
26     if (startIndex == -1) {
27         free(result);
28         return NULL; // 起始顶点不存在
29     }
30
31     // 创建队列用于BFS
32     int queue[MAX_VERTEX_NUM];
33     int front = 0, rear = 0;
34
35     // 将起始顶点入队
36     queue[rear++] = startIndex;
37     visited[startIndex] = 1;
38     distance[startIndex] = 0;
39     result[count++] = G.vertices[startIndex].data;
40
41     // BFS遍历
42     while (front < rear) {
43         // 出队一个顶点
44         int current = queue[front++];
45
46         // 如果当前顶点的距离已经等于k-1, 那么它的邻接点距离将是k
47         // , 不需要继续遍历
48         if (distance[current] == k - 1) {
49             continue;
50         }
51
52         // 访问所有邻接点
53         ArcNode *p = G.vertices[current].firstarc;
54         while (p != NULL) {
55             int adjIndex = p->adjvex;
56             if (!visited[adjIndex]) {
57                 visited[adjIndex] = 1;
58                 distance[adjIndex] = distance[current] + 1;
59
60                 // 添加到结果集
61                 result[count++] = G.vertices[adjIndex].data;
62
63                 // 只有当距离小于k-1的顶点才入队, 以便继续遍历其
64                 // 邻接点
65                 if (distance[adjIndex] < k - 1) {
66                     queue[rear++] = adjIndex;
67                 }
68             }
69             p = p->nextarc;
70         }
71     }
72 }

```



```

66         }
67         p = p->nextarc;
68     }
69 }
70
71 // 返回结果数组的一个副本，只包含需要的部分
72 VertexType *finalResult = (VertexType *)malloc((count + 1) *
73 sizeof(VertexType));
74 for (i = 0; i < count; i++) {
75     finalResult[i] = result[i];
76 }
77 finalResult[count].key = -1; // 设置结束标记
78 free(result);
79
80 return finalResult;
81 }
82
83 //Dijkstra algorithm
84 int ShortestPath(ALGraph G, KeyType u, KeyType v)
85 {
86     int i, j, min, minIndex;
87     int visited[MAX_VERTEX_NUM] = {0};
88     int dist[MAX_VERTEX_NUM];
89
90     int startIndex = GetLocation(G, u);
91     int endIndex = GetLocation(G, v);
92     if(startIndex == -1 || endIndex == -1) return -1; // 起始或结
93     束顶点不存在
94
95     if(startIndex == endIndex) return 0; // 起始和结束顶点相同
96
97     for(i = 0; i < G.vexnum; i++)
98     {
99         dist[i] = 32767;
100     }
101
102     dist[startIndex] = 0; // 起始顶点到自身的距离为0
103
104     for(i = 0; i < G.vexnum; i++)
105     {
106         min = 32767;
107         minIndex = -1;
108
109         for(j = 0; j < G.vexnum; j++)
110         {
111             if(!visited[j] && dist[j] < min)
112             {
113                 min = dist[j];
114                 minIndex = j;
115             }
116         }
117
118         if(minIndex == -1) break; // 所有可达顶点都已访问
119
120         visited[minIndex] = 1; // 标记当前顶点为已访问
    
```

```

119         if(minIndex == endIndex) return dist[minIndex]; // 找到目
120         标顶点
121
122         ArcNode *p = G.vertices[minIndex].firstarc;
123         while(p != NULL)
124         {
125             int adjIndex = p -> adjvex;
126
127             int weight = 1;
128
129             if(!visited[adjIndex] && dist[minIndex] != 32767 &&
130                 dist[minIndex] + weight < dist[adjIndex])
131             {
132                 dist[adjIndex] = dist[minIndex] + weight;
133             }
134
135             p = p -> nextarc;
136         }
137     }
138
139     if(dist[endIndex] == 32767)
140     {
141         return -1; // 目标顶点不可达
142     }
143
144     return dist[endIndex]; // 返回最短路径长度
145 }
146
147
148 int ConnectedComponentsNum(ALGraph G)
149 {
150     int i, j, count = 0;
151     int visited[MAX_VERTEX_NUM] = {0};
152
153     for(i = 0; i < G.vexnum; i++)
154     {
155         if(!visited[i])
156         {
157             count++;
158             // 深度优先搜索遍历当前连通分量
159             DFS(G, i, visited, NULL);
160         }
161     }
162
163     return count;
164 }
165
166 void PrintGraphList(ALGraph G)
167 {
168     int i;
169     ArcNode *p;
170
171     for(i = 0; i < G.vexnum; i++)
172     {

```

```
173     printf("Vertex %d (%s): ", G.vertices[i].data.key, G.vertices[i].data.others);
174     p = G.vertices[i].firstarc;
175     while(p != NULL)
176     {
177         printf("%d ", G.vertices[p->adjvex].data.key);
178         p = p->nextarc;
179     }
180     printf("\n");
181 }
182 }
```

Listing 52 Graph-Addons.h

Graph-Multi.h

```
1 status InitGraphGroup(GraphGroup &Ggroup)
2 // 初始化图组
3 {
4     Ggroup.Gnum = 0;
5     for(int i = 0; i < MAX_VERTEX_NUM; i++) {
6         Ggroup.name[i][0] = '\0'; // 初始化名称为空字符串
7         Ggroup.Ggroup[i].vexnum = 0; // 初始化图为空
8     }
9     return OK;
10 }
11
12 status AddGraph(GraphGroup &Ggroup, char name[])
13 {
14     if (Ggroup.Gnum >= MAX_VERTEX_NUM) {
15         return ERROR; // 图组已满
16     }
17
18     strcpy(Ggroup.name[Ggroup.Gnum], name);
19     Ggroup.Gnum++;
20
21     return OK;
22 }
23
24 status RemoveGraph(GraphGroup &Ggroup, char name[])
25 {
26     for(int i = 0; i < Ggroup.Gnum; i++) {
27         if (strcmp(Ggroup.name[i], name) == 0) {
28             // 找到图, 删除
29             for (int j = i; j < Ggroup.Gnum - 1; j++) {
30                 strcpy(Ggroup.name[j], Ggroup.name[j + 1]);
31                 Ggroup.Ggroup[j] = Ggroup.Ggroup[j + 1];
32             }
33             Ggroup.Gnum--;
34             return OK;
35         }
36     }
```

```
37     return ERROR; // 未找到图
38 }
39
40
41 int LocateGraph(GraphGroup Ggroup, char name[])
42 {
43     for (int i = 0; i < Ggroup.Gnum; i++) {
44         if (strcmp(Ggroup.name[i], name) == 0) {
45             return i; // 找到图的索引
46         }
47     }
48
49     return -1; // 未找到
50 }
51
52 status PrintGraphGroup(GraphGroup Ggroup)
53 {
54     for (int i = 0; i < Ggroup.Gnum; i++) {
55         printf("Graph %d: %s\n", i, Ggroup.name[i]);
56         // 可以添加更多的打印信息, 例如顶点和边的信息
57     }
58
59     return OK;
60 }
```

Listing 53 Graph-Multi.h

main.cpp

```
1  #include "def.h"
2  #include "Graph.h"
3  #include "Graph-Multi.h"
4  #include "Graph-Addons.h"
5  #include "Graph-IO.h"
6
7  int main()
8  {
9      int mode = 1, op = 1, count = 0, temp = 0; // 操作码
10     ALGraph *G_operated = NULL;
11     ALGraph G_single; // 单棵图指针
12     GraphGroup Graphs;
13     InitGraphGroup(Graphs); // 初始化多棵图
14
15     VertexType V[30];
16     KeyType VR[100][2];
17
18     char fileName[30];
19     char graphName[20];
20
21     int graph_idx = 0;
22     int i, j, k, flag = 1;
23     int first, next;
```

```

24     int loc = -1, loc1 = -1, loc_idx = -1, pathLength;
25
26     KeyType key, key1, key2;
27     VertexType v;
28     VertexType *result;
29
30     while(mode)
31     {
32         system("cls");      printf("\n\n");
33         printf("          Menu for Mode Selection \n");
34         printf("-----\n");
35         printf("          1. Single Graph \n");
36         printf("          2. Multi Graphs \n");
37         printf("          0. Exit \n");
38         printf("-----\n");
39         printf("      Choose the mode from 0 to 2: ");
40         scanf("%d", &mode);
41         G_operated = NULL; // 重置操作的链表指针
42
43         while(op)
44         {
45
46             if(mode == 2)
47             {
48                 system("cls");      printf("\n\n");
49                 printf("-----\n");
50                 printf("          Multi-BiTrees Operation Mode:\n");
51                 printf("-----\n");
52                 if(graph_idx >= Graphs.Gnum)
53                 {
54                     G_operated = NULL;
55                     printf("          No graph\n");
56                     printf("-----\n");
57                 }
58                 else
59                 {
60                     G_operated = &Graphs.Ggroup[graph_idx];
61                     printf("          Current list: %s",
62                           Graphs.name[graph_idx]);
63                     printf("          List id: %d", graph_idx + 1);
64                     if(Graphs.Ggroup[graph_idx].vexnum == 0)
65                         printf(" (Empty)");
66                     printf("\n-----\n");
67                 }
68             }
69             else if(mode == 1)
70             {

```

```

70         G_operated = &G_single;
71         system("cls");      printf("\n\n");
72         printf("-----\n");
73         printf("          Single-Graph Operation Mode:\n"
);
74         printf("-----\n");
75     }
76     else if(mode == 0)
77     {
78         printf("Exiting...\n");
79         break;
80     }
81     else
82     {
83         printf("Invalid mode!\n");
84         continue;
85     }
86
87     printf("          Menu for Operation Selection \n");
88     printf("-----\n");
89     printf("          1. CreateGraph          7.
InsertVex\n");
90     printf("          2. DestroyGraph         8.
DeleteVex\n");
91     printf("          3. GetLocation          9.
InsertArc\n");
92     printf("          4. PutVex              10.
DeleteArc\n");
93     printf("          5. FirstAdjVex          11.
DFSTraverse\n");
94     printf("          6. NextAdjVex          12.
BFSTraverse\n");
95     printf("-----\n");
96     printf("          File Operation Menu \n");
97     printf("-----\n");
98     printf("          13. SaveGraph           14.
LoadGraph\n");
99     printf("-----\n");
100    printf("          Additional Functions Menu \n");
101    printf("-----\n");
102    printf("          15. VerticesSetLessThanK 17.
ConnectedComponentsNum\n");
103    printf("          16. ShortestPath          18.
PrintGraphList\n");
104    printf("-----\n");
105
106    if(mode == 2)

```

```

107         {
108             printf("          Multi-BiTrees Operation
Selection \n");
109             printf("-----\n");
110             printf("          19. AddGraph          22.
PrintGraphGroup\n");
111             printf("          20. RemoveGraph       23.
SwitchGraph\n");
112             printf("          21. LocateGraph       24.
InitGraphGroup \n");
113             printf("-----\n");
114         }
115
116         printf("          0.Exit\n");
117         printf("-----\n\n");
118         printf("          Choose your operations [0~%d]: ", mode ==
2 ? 25 : 14);
119
120         scanf("%d", &op);
121         if(op == 0)
122         {
123             printf("Returning to Main Menu...\n");
124             op = 1;
125             break;
126         }
127
128         if(mode == 2 && op != 19 && op != 22 && op != 23 &&
Graphs.Gnum <= graph_idx)
129         {
130             printf("No graph exist! Please create a tree
first.\n");
131             op = 1; // 重置操作码
132             getchar(); // 清除缓冲区中的换行符
133             getchar(); // 等待用户输入
134             continue;
135         }
136
137         switch(op)
138         {
139             case 1:
140                 printf("Please input the Graph series:\n");
141                 i = 0;
142                 do {
143                     scanf("%d%s",&V[i].key,V[i].others);
144                 } while(V[i++].key!=-1);
145                 i=0;
146                 do {
147                     scanf("%d%d",&VR[i][0],&VR[i][1]);
148                 } while(VR[i++][0]!=-1);
149                 VR[i][0] = -1;
150                 VR[i][1] = -1;
151                 if(CreateCraph(*G_operated,V,VR) == ERROR)

```

```

152         {
153             printf("Graph creation failed!\n");
154         }
155         else
156             printf("Graph created successfully!\n");
157         break;
158
159     case 2:
160         if(DestroyGraph(*G_operated) == ERROR)
161         {
162             printf("Graph destruction failed!\n");
163         }
164         else
165             printf("Graph destroyed successfully!\n");
166     ;
167
168         break;
169
170     case 3:
171         printf("Please input the key of the vertex to
172         be located:\n");
173         scanf("%d",&key);
174         loc = GetLocation(*G_operated, key);
175         if(loc == -1)
176             printf("Vertex not found!\n");
177         else
178         {
179             printf("Vertex found at index: %d\n", loc
180 );
181             printf("Vertex information: Key: %d,
182 Others: %s\n", G_operated->vertices[loc].data.key, G_operated-
183 >vertices[loc].data.others);
184         }
185         break;
186
187     case 4:
188         printf("Please input the key of the vertex to
189         be modified:\n");
190         scanf("%d",&key);
191         printf("Please input the new information of
192         the vertex:\n");
193         scanf("%d %s", &v.key, v.others);
194         if(PutVex(*G_operated, key, v) == ERROR)
195             printf("Vertex modification failed!\n");
196         else
197             printf("Vertex modified successfully!\n");
198     ;
199
200         break;
201
202     case 5:
203         printf("Please input the key of the vertex to
204         be found:\n");
205         scanf("%d",&key);
206         first = FirstAdjVex(*G_operated, key);
207         if(first == -1)
208             printf("No adjacent vertex found!\n");

```



```

198         else
199         {
200             printf("First adjacent vertex: %d\n",
first);
201             printf("Adjacent vertices: %d, %s",
G_operated->vertices[first].data.key, G_operated->vertices[
first].data.others);
202         }
203
204         break;
205
206         case 6:
207             printf("Please input the key of the vertex to
be found:\n");
208             scanf("%d %d",&key1,&key2);
209             next = NextAdjVex(*G_operated, key1,key2);
210             if(next == -1)
211                 printf("No next adjacent vertex found!\n"
);
212             else
213             {
214                 printf("Next adjacent vertex: %d\n", next
);
215                 printf("Adjacent vertices: %d, %s",
G_operated->vertices[next].data.key, G_operated->vertices[next
].data.others);
216             }
217
218             break;
219
220         case 7:
221             printf("Please input the key of the vertex to
be inserted:\n");
222             scanf("%d",&v.key);
223             printf("Please input the other information of
the vertex:\n");
224             scanf("%s",v.others);
225             if(GetLocation(*G_operated, v.key) != -1)
226             {
227                 printf("Vertex already exists!\n");
228                 break;
229             }
230             if(InsertVex(*G_operated, v) == ERROR)
231                 printf("Vertex insertion failed!\n");
232             else
233                 printf("Vertex inserted successfully!\n");
234
235             break;
236
237         case 8:
238             printf("Please input the key of the vertex to
be deleted:\n");
239             scanf("%d",&key);
240             if(DeleteVex(*G_operated, key) == ERROR)
                printf("Vertex deletion failed!\n");

```

```

241         else
242             printf("Vertex deleted successfully!\n");
243             break;
244
245         case 9:
246             printf("Please input the keys of the vertices
to be connected:\n");
247             scanf("%d%d",&key,&key1);
248             if(InsertArc(*G_operated, key, key1) == ERROR
)
249                 printf("Edge insertion failed!\n");
250             else
251                 printf("Edge inserted successfully!\n");
252             break;
253
254         case 10:
255             printf("Please input the keys of the vertices
to be disconnected:\n");
256             scanf("%d%d",&key,&key1);
257             if>DeleteArc(*G_operated, key, key1) == ERROR
)
258                 printf("Edge deletion failed!\n");
259             else
260                 printf("Edge deleted successfully!\n");
261             break;
262
263         case 11:
264             printf("Please input the key of the vertex to
be traversed:\n");
265             scanf("%d",&key);
266             if(DFS_Traverse(*G_operated, PrintVertex) ==
ERROR)
267                 printf("DFS traversal failed!\n");
268             else
269                 printf("DFS traversal completed
successfully!\n");
270             break;
271
272         case 12:
273             printf("Please input the key of the vertex to
be traversed:\n");
274             scanf("%d",&key);
275             if(BFS_Traverse(*G_operated, PrintVertex) ==
ERROR)
276                 printf("BFS traversal failed!\n");
277             else
278                 printf("BFS traversal completed
successfully!\n");
279             break;
280
281         case 13:
282             printf("Please input the file name to save
the graph:\n");
283             scanf("%s",fileName);
284             if(SaveGraph(*G_operated, fileName) == ERROR)

```

```
285         printf("Graph saving failed!\n");
286     else
287         printf("Graph saved successfully!\n");
288     break;
289
290     case 14:
291         printf("Please input the file name to load
the graph:\n");
292         scanf("%s",fileName);
293         if(LoadGraph(*G_operated, fileName) == ERROR)
294             printf("Graph loading failed!\n");
295         else
296             printf("Graph loaded successfully!\n");
297         break;
298
299     case 15:
300         printf("Please input the key of the vertex
and the distance:\n");
301         scanf("%d%d",&key,&k);
302         result = VerticesSetLessThanK(*G_operated,
key, k);
303         if(result == NULL)
304             printf("No vertices found!\n");
305         else
306         {
307             printf("Vertices within distance %d from
vertex %d:\n", k, key);
308             for(i = 0; result[i].key != -1; i++)
309                 printf("%d %s\n", result[i].key,
result[i].others);
310         }
311         break;
312
313     case 16:
314         printf("Please input the start and end
vertices:\n");
315         scanf("%d%d",&key,&key1);
316         pathLength = ShortestPath(*G_operated, key,
key1);
317         if(pathLength == -1)
318             printf("No path found!\n");
319         else
320             printf("Shortest path length: %d\n",
pathLength);
321         break;
322
323     case 17:
324         printf("Number of connected components: %d\n"
, ConnectedComponentsNum(*G_operated));
325         break;
326
327     case 18:
328         printf("Graph adjacency list:\n");
329         PrintGraphList(*G_operated);
330         break;
```

```

331
332         case 19:
333             printf("Please input the name of the graph to
be added:\n");
334             scanf("%s",fileName);
335             if(AddGraph(Graphs, fileName) == ERROR)
336                 printf("Graph addition failed!\n");
337             else
338                 printf("Graph added successfully!\n");
339             break;
340
341         case 20:
342             printf("Please input the name of the graph to
be removed:\n");
343             scanf("%s",fileName);
344             if(RemoveGraph(Graphs, fileName) == ERROR)
345                 printf("Graph removal failed!\n");
346             else
347                 printf("Graph removed successfully!\n");
348
349         case 21:
350             printf("Please input the name of the graph to
be located:\n");
351             scanf("%s",fileName);
352             loc_idx = LocateGraph(Graphs, fileName);
353             if(loc_idx == -1)
354                 printf("Graph not found!\n");
355             else
356                 printf("Graph found at index: %d\n",
loc_idx);
357             break;
358
359         case 22:
360             printf("Graph group:\n");
361             PrintGraphGroup(Graphs);
362             break;
363
364         case 23:
365             printf("Please input the name of the graph to
be switched:\n");
366             scanf("%s",fileName);
367             graph_idx = LocateGraph(Graphs, fileName);
368             if(graph_idx == -1)
369                 printf("Graph not found!\n");
370             else
371                 printf("Switched to graph at index: %d\n"
, graph_idx);
372             break;
373
374         case 24:
375             printf("Initializing graph group...\n");
376             if(InitGraphGroup(Graphs) == ERROR)
377                 printf("Graph group initialization failed
!\n");
378             else

```

```
379         printf("Graph group initialized\n");
380         break;
381     }
382
383     printf("Press any key to continue...");
384     getchar(); // 清除缓冲区中的换行符
385     getchar(); // 等待用户输入
386 }
387
388 op = 1; // 重置操作码
389
390 }
391
392 return 0; // 返回0表示程序正常结束
393 }
394 }
```

Listing 54 main.cpp